EN.553.688 Computing for Applied Math

Jonathan Y. Ma

June 2026

This course was taught by Dr. Daniel Naiman in the Fall 2025 Semester. The notes are transcribed by Jonathan Ma and may contain errors and omitted content, so any issues are to be directed towards me.

Contents

I Applied Mathematics **7**

1 The Discrete Fourier Transform and Fast Fourier Transform **8**
1.1 The Discrete Fourier Transform . . . 8
1.1.1 Matrix Representation . . . 8
1.1.2 Periodicity . . . 8
1.2 The Inverse Transform . . . 9
1.3 Time and Frequency Domain Interpretation . . . 10
1.4 The Fast Fourier Transform . . . 10
1.4.1 Even–Odd Decomposition . . . 10
1.4.2 Matrix Factorization . . . 11
1.4.3 Recursive Structure . . . 11
1.4.4 Complexity . . . 11
1.5 Exercises . . . 11

2 The Delta Method **12**
2.1 Overview . . . 12
2.2 One-Dimensional Delta Method . . . 12
2.3 Examples . . . 13
2.4 Two-Dimensional Delta Method . . . 14
2.5 Ratio Example . . . 14
2.6 Functions of Sample Means . . . 15

3 Monte Carlo Methods **16**
3.1 Overview . . . 16
3.2 Monte Carlo Estimation . . . 16
3.3 Random Number Generation . . . 17
3.4 The Inversion Method . . . 17
3.5 Acceptance–Rejection Sampling . . . 18
3.6 Example: Sampling a Standard Normal . . . 19
3.7 Examples of Monte Carlo Problems . . . 19
3.8 Summary . . . 20

4 Markov Chains **21**
4.1 Sampling Dependent Random Variables . . . 21

4.1.1 Sampling Using a Single Variable . . . 21
4.1.2 Conditional Sampling . . . 21
4.2 Sampling Multiple Dependent Variables . . . 22
4.3 The Markov Assumption . . . 22
4.4 Markov Chains . . . 23
4.4.1 Transition Matrices . . . 23
4.5 Examples of Markov Chains . . . 24
4.6 One-Step Distributions . . . 25
4.7 Multi-Step Transition Probabilities . . . 25
4.8 Irreducibility . . . 26
4.9 Periodicity . . . 27
4.10 Recurrence and Transience . . . 27
4.11 Stationary Distributions . . . 28
4.12 Ergodicity . . . 28
4.13 Summary . . . 29

5 The Metropolis–Hastings Algorithm 30
5.1 Overview . . . 30
5.2 Self-Avoiding Paths . . . 30
5.3 Uniform Sampling of Self-Avoiding Paths . . . 31
5.4 Number of Bends . . . 31
5.5 Detailed Balance . . . 31
5.6 Metropolis–Hastings Construction . . . 32
5.6.1 Uniform Neighbor Proposal . . . 32
5.6.2 Weighted Neighbor Proposal . . . 33
5.7 Metropolis–Hastings for Uniform Self-Avoiding Paths . . . 34
5.8 Autocorrelation and Convergence Diagnostics . . . 34
5.9 Nonuniform Target Distributions . . . 35
5.10 Ergodicity and Convergence . . . 35
5.11 Summary . . . 36

6 Brownian Motion 37
6.1 From Random Walk to Brownian Motion . . . 37
6.2 Definition and Basic Properties . . . 37
6.2.1 Normality of Increments . . . 38
6.3 Simulating Brownian Motion . . . 38
6.4 Brownian Motion with Drift and Volatility . . . 39
6.5 Fitting Brownian Motion with Drift and Volatility . . . 39
6.6 Geometric Brownian Motion . . . 40
6.7 Higher-Dimensional Brownian Motion . . . 41
6.8 Summary . . . 42

7 Stochastic Differential Equations 43
7.1 Ordinary Differential Equations . . . 43
7.2 Euler's Method . . . 43

7.3 A One-Dimensional ODE Example . . . 44
7.4 Higher-Dimensional ODEs . . . 45
7.4.1 A Two-Dimensional Rotating and Contracting System . . . 45
7.5 Brownian Motion Recap . . . 46
7.6 Stochastic Differential Equations . . . 47
7.7 Euler–Maruyama Method . . . 47
7.8 Ito's Lemma . . . 48
7.9 Brownian Motion with Drift . . . 49
7.10 Geometric Brownian Motion . . . 49
7.11 Log Transform of Geometric Brownian Motion . . . 50
7.12 Calibration of Geometric Brownian Motion . . . 50
7.13 Summary . . . 52

8 Directional Derivatives **54**
8.1 Definition . . . 54
8.2 A Direction-Dependent Example . . . 54
8.2.1 Partial Derivative with Respect to x . . . 54
8.2.2 Partial Derivative with Respect to y . . . 55
8.3 Continuity and Differentiability . . . 56
8.4 Directional Derivatives at the Origin . . . 57

9 Mean-Reverting Processes **59**
9.1 Motivation . . . 59
9.2 Euler–Maruyama Simulation . . . 59
9.3 Exact Conditional Distribution . . . 60
9.4 Exact Simulation on an Arbitrary Time Grid . . . 61
9.5 Long-Run Behavior . . . 62
9.6 Covariance Structure . . . 62
9.7 Likelihood for Observed Data . . . 63
9.8 Simplifying Notation . . . 63
9.9 Maximum Likelihood Estimation . . . 63
9.9.1 Estimating μ for Fixed θ . . . 64
9.9.2 Estimating σ for Fixed θ and μ . . . 64
9.9.3 Grid Search over θ . . . 64
9.10 Parametric Bootstrap Confidence Intervals . . . 65
9.11 Summary . . . 65

10 Boosting Methods **66**
10.1 Empirical Risk Minimization . . . 66
10.2 Examples of Loss Functions . . . 66
10.3 Weak Learners and Regularity . . . 67
10.4 Gradient Boosting . . . 68
10.5 Gradient Boosting Algorithm . . . 69
10.6 Least Squares Gradient Boosting . . . 69
10.7 Weak Learner Complexity . . . 70

10.8 AdaBoost . . . 70
10.9 AdaBoost and Exponential Loss . . . 72
10.10Summary . . . 72

II Scientific Computing 74

11 Python Fundamentals 75
11.1 Assignment . . . 75
11.1.1 Multiple Assignment . . . 75
11.1.2 Unpacking . . . 76
11.2 Basic Data Types . . . 76
11.2.1 Numbers . . . 76
11.2.2 Booleans . . . 77
11.2.3 Lists . . . 77
11.2.4 Tuples . . . 77
11.2.5 Dictionaries . . . 78
11.2.6 Sets . . . 78
11.3 Evaluation and Control Flow . . . 78
11.3.1 Conditional Statements . . . 78
11.3.2 Loops . . . 79
11.3.3 List Comprehensions . . . 79
11.4 Functions . . . 79
11.4.1 Default Arguments . . . 79
11.4.2 Keyword Arguments . . . 80
11.4.3 Variable-Length Arguments . . . 80
11.5 Timing Code . . . 80
11.5.1 Using `time` . . . 80
11.5.2 Using `timeit` . . . 80
11.6 Debugging Code . . . 81
11.6.1 Common Error Types . . . 81
11.6.2 Print Debugging . . . 81
11.6.3 Assertions . . . 81
11.6.4 Using `pdb` . . . 82
11.7 String Manipulation . . . 82
11.7.1 Common String Methods . . . 82
11.7.2 Splitting and Joining . . . 82
11.7.3 Formatted Strings . . . 83
11.7.4 String Membership . . . 83
11.8 Classes . . . 83
11.8.1 The `self` Argument . . . 83
11.8.2 Instance Attributes and Class Attributes . . . 84
11.9 Overloaded Operators . . . 84
11.9.1 String Representation . . . 84
11.9.2 Addition . . . 84

11.9.3 Scalar Multiplication . . . 85
11.9.4 Length and Indexing . . . 85
11.10 Decorators . . . 86
11.10.1 Basic Decorator . . . 86
11.10.2 Decorators with Arguments . . . 86
11.10.3 Timing Decorator . . . 87
11.10.4 Using `functools.wraps` . . . 87
11.11 Class Decorators and Properties . . . 88
11.12 Static Methods and Class Methods . . . 88
11.13 Summary . . . 89

12 Advanced Python Topics for Data Science 90
12.1 Web Scraping and Regular Expressions . . . 90
12.1.1 Web Scraping Overview . . . 90
12.1.2 Requests + BeautifulSoup . . . 90
12.1.3 Finding Elements . . . 90
12.1.4 Using CSS Selectors . . . 91
12.1.5 Regular Expressions . . . 91
12.1.6 Common Patterns . . . 91
12.1.7 Extraction Example . . . 92
12.2 Parallel Computing . . . 92
12.2.1 Multiprocessing . . . 92
12.2.2 Parallel Loops . . . 92
12.2.3 Threading . . . 92
12.3 Priority Queues and Trees . . . 93
12.3.1 Priority Queue (Heap) . . . 93
12.3.2 Custom Priority Queue . . . 93
12.3.3 Binary Tree Implementation . . . 93
12.3.4 Traversal (Inorder) . . . 94
12.3.5 Binary Search Tree Insert . . . 94
12.4 PyTorch . . . 94
12.4.1 Tensors . . . 94
12.4.2 Automatic Differentiation . . . 94
12.4.3 Neural Network Example . . . 94
12.4.4 Training Loop . . . 95
12.5 Pandas . . . 95
12.5.1 DataFrame Creation . . . 95
12.5.2 Basic Operations . . . 95
12.5.3 Filtering . . . 96
12.5.4 Adding Columns . . . 96
12.5.5 GroupBy . . . 96
12.5.6 Merging DataFrames . . . 96
12.5.7 Reading and Writing Files . . . 96
12.6 Summary . . . 96

13 Neural Networks **97**
13.1 Architecture Overview . . . 97
13.2 Layer Equations . . . 97
13.2.1 First Hidden Layer . . . 97
13.2.2 Second Hidden Layer . . . 98
13.2.3 Output Layer (Pre-Softmax) . . . 98
13.3 Parameter Counting . . . 98
13.4 Softmax Output Layer . . . 98
13.5 Cross-Entropy Loss . . . 98
13.6 Gradient Descent . . . 99
13.6.1 Backpropagation Idea . . . 99
13.7 Interpretation . . . 99
13.7.1 Decision Regions . . . 99
13.7.2 Training Dynamics . . . 99
13.7.3 Probabilistic Output . . . 99
13.8 PyTorch Implementation . . . 99
13.8.1 Model Definition . . . 99
13.8.2 Loss and Optimizer . . . 100
13.8.3 Training Loop . . . 100
13.9 Mini-Batch Training . . . 100
13.10Regularization . . . 101
13.11Summary . . . 101

Part I

Applied Mathematics

Chapter 1

The Discrete Fourier Transform and Fast Fourier Transform

1.1 The Discrete Fourier Transform

Definition 1.1.1. Let $x = (x_0, \ldots, x_{N-1}) \in \mathbb{C}^N$. The *Discrete Fourier Transform (DFT)* of x is the vector $\hat{x} \in \mathbb{C}^N$ defined by

$$\hat{x}_k = \frac{1}{\sqrt{N}} \sum_{j=0}^{N-1} x_j e^{-2\pi i k j/N}, \quad k = 0, \ldots, N-1.$$

The DFT is a linear transformation mapping the time domain representation x to its frequency domain representation $\hat{x}$.

1.1.1 Matrix Representation

Proposition 1.1.1. *The DFT can be written as a matrix-vector multiplication*

$$\hat{x} = \frac{1}{\sqrt{N}} M_N x,$$

where $M_N \in \mathbb{C}^{N \times N}$ has entries

$$(M_N)_{k,j} = e^{-2\pi i k j/N}.$$

Proof. This follows directly from the definition of the DFT by identifying the coefficients multiplying each x_j in the sum defining $\hat{x}_k$. □

1.1.2 Periodicity

Proposition 1.1.2. *The DFT is periodic in the frequency index:*

$$\hat{x}_{k+N} = \hat{x}_k \quad \textit{for all } k \in \mathbb{Z}.$$

Proof.

$$\hat{x}_{k+N} = \frac{1}{\sqrt{N}} \sum_{j=0}^{N-1} x_j e^{-2\pi i(k+N)j/N} = \frac{1}{\sqrt{N}} \sum_{j=0}^{N-1} x_j e^{-2\pi ikj/N} e^{-2\pi ij}.$$

Since $e^{-2\pi ij} = 1$ for all integers j, the result follows. □

Thus $\hat{x}_k = \hat{x}_{k \bmod N}$, so frequencies are defined modulo N.

1.2 The Inverse Transform

Definition 1.2.1. The inverse DFT (IDFT) is given by

$$x_k = \frac{1}{\sqrt{N}} \sum_{j=0}^{N-1} \hat{x}_j e^{2\pi ikj/N}, \quad k = 0, \ldots, N-1.$$

Theorem 1.2.1. *The DFT is unitary. In particular,*

$$M_N^{-1} = M_N^*,$$

and the inverse transform recovers the original vector:

$$x = \frac{1}{\sqrt{N}} M_N^* \hat{x}.$$

Proof. Consider the inner product of two columns k and ℓ of M_N:

$$\sum_{j=0}^{N-1} e^{-2\pi ikj/N} \overline{e^{-2\pi i\ell j/N}} = \sum_{j=0}^{N-1} e^{-2\pi i(k-\ell)j/N}.$$

If $k = \ell$, this equals N. Otherwise it is a geometric series:

$$\sum_{j=0}^{N-1} r^j = \frac{1 - r^N}{1 - r} = 0,$$

since $r = e^{-2\pi i(k-\ell)/N} \neq 1$ but $r^N = 1$.

Thus the columns are orthogonal and have norm $\sqrt{N}$, so

$$\frac{1}{\sqrt{N}} M_N$$

is unitary, proving the result. □

1.3 Time and Frequency Domain Interpretation

Definition 1.3.1. For $k \in \{0, \dots, N-1\}$, define the discrete complex sinusoid

$$\mathrm{DCS}(k)_j = e^{2\pi i jk/N}, \quad j = 0, \dots, N-1.$$

Proposition 1.3.1. *The inverse DFT matrix can be written as*

$$Q_N = [DCS(0), \dots, DCS(N-1)].$$

Corollary 1.3.1. *Every signal admits a frequency decomposition:*

$$x = \sum_{k=0}^{N-1} \hat{x}_k \, DCS(k).$$

Remark 1.3.1. This shows that the DFT expresses x as a superposition of orthogonal frequency components. The coefficient $\hat{x}_k$ measures the contribution of frequency k/N.

1.4 The Fast Fourier Transform

1.4.1 Even–Odd Decomposition

Let

$$x_E = (x_0, x_2, \dots, x_{N-2}), \quad x_O = (x_1, x_3, \dots, x_{N-1}).$$

Proposition 1.4.1. *The DFT satisfies the recursion*

$$\hat{x}_k = \hat{x}_E[k] + e^{-2\pi ik/N} \hat{x}_O[k], \quad k = 0, \dots, \frac{N}{2} - 1,$$

and

$$\hat{x}_{k+N/2} = \hat{x}_E[k] - e^{-2\pi ik/N} \hat{x}_O[k].$$

Proof. Split the sum defining $\hat{x}_k$ into even and odd indices:

$$\hat{x}_k = \sum_{j=0}^{N/2-1} x_{2j} e^{-2\pi ik(2j)/N} + \sum_{j=0}^{N/2-1} x_{2j+1} e^{-2\pi ik(2j+1)/N}.$$

Factor:

$$= \sum_{j=0}^{N/2-1} x_E[j] e^{-2\pi ikj/(N/2)} + e^{-2\pi ik/N} \sum_{j=0}^{N/2-1} x_O[j] e^{-2\pi ikj/(N/2)}.$$

Recognizing DFTs of length $N/2$ gives the result. □

1.4.2 Matrix Factorization

Proposition 1.4.2. *Let B_N be the permutation matrix that maps*

$$(x_0, x_1, \ldots, x_{N-1}) \mapsto (x_0, x_2, \ldots, x_{N-2}, x_1, x_3, \ldots, x_{N-1}),$$

and let $A_{N/2} = \mathrm{diag}(e^{-2\pi ik/N})$. Then

$$M_N = \begin{bmatrix} I_{N/2} & A_{N/2} \\ I_{N/2} & -A_{N/2} \end{bmatrix} \begin{bmatrix} M_{N/2} & 0 \\ 0 & M_{N/2} \end{bmatrix} B_N.$$

This factorization reduces a size-N DFT into two size-$N/2$ DFTs.

1.4.3 Recursive Structure

By recursively applying the factorization, the DFT reduces to repeated applications of

$$M_2 = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}.$$

1.4.4 Complexity

Theorem 1.4.1. *The FFT computes the DFT in $\mathcal{O}(N \log N)$ operations.*

Proof. At each recursion level, the problem splits into two subproblems of size $N/2$, and combining them requires $\mathcal{O}(N)$ operations. The recursion depth is $\log_2 N$, giving total cost $\mathcal{O}(N \log N)$. □

Remark 1.4.1. The efficiency arises from the sparsity of intermediate matrices: each row contains only two nonzero entries, so matrix-vector multiplication is linear in N.

1.5 Exercises

Exercise 1.5.1. Compute the DFT explicitly for small vectors:

1. Derive a closed-form expression for the DFT of a vector in $\mathbb{C}^2$.
2. Derive the DFT matrix M_4 and compute $\hat{x}$ for a general vector $x \in \mathbb{C}^4$.

Chapter 2

The Delta Method

2.1 Overview

The delta method approximates moments of smooth functions of random variables. If X is concentrated near its mean μ_X and f is smooth near μ_X, then $f(X)$ is concentrated near $f(\mu_X)$.

2.2 One-Dimensional Delta Method

Let X have mean μ_X and variance σ_X^2. Suppose $f : \mathbb{R} \to \mathbb{R}$ is twice continuously differentiable near μ_X.

Using a second-order Taylor expansion around μ_X,

$$f(X) \approx f(\mu_X) + f'(\mu_X)(X - \mu_X) + \frac{1}{2} f''(\mu_X)(X - \mu_X)^2.$$

Taking expectations gives

$$\mathbb{E}[f(X)] \approx f(\mu_X) + \frac{1}{2} f''(\mu_X)\sigma_X^2.$$

Proposition 2.2.1. *If X is concentrated near μ_X, then*

$$\mathbb{E}[f(X)] \approx f(\mu_X) + \frac{1}{2} f''(\mu_X)\mathrm{Var}(X).$$

Proof. Taking expectations in the second-order Taylor approximation,

$$\mathbb{E}[f(X)] \approx f(\mu_X) + f'(\mu_X)\mathbb{E}[X - \mu_X] + \frac{1}{2} f''(\mu_X)\mathbb{E}[(X - \mu_X)^2].$$

Since $\mathbb{E}[X - \mu_X] = 0$ and $\mathbb{E}[(X - \mu_X)^2] = \sigma_X^2$, the result follows. □

For the variance, use the first-order Taylor approximation:

$$f(X) \approx f(\mu_X) + f'(\mu_X)(X - \mu_X).$$

Thus

$$\mathrm{Var}(f(X)) \approx [f'(\mu_X)]^2 \sigma_X^2.$$

Proposition 2.2.2. *The first-order delta method gives*

$$\mathrm{Var}(f(X)) \approx [f'(\mu_X)]^2 \mathrm{Var}(X).$$

2.3 Examples

Example 2.3.1. Let R be a random radius with mean μ_R and variance σ_R^2. The area of a circle is

$$A = f(R) = \pi R^2.$$

Since $f'(r) = 2\pi r$ and $f''(r) = 2\pi$,

$$\mathbb{E}[A] \approx \pi \mu_R^2 + \pi \sigma_R^2.$$

This is exact because

$$\mathbb{E}[R^2] = \mathrm{Var}(R) + (\mathbb{E}[R])^2.$$

The approximate variance is

$$\mathrm{Var}(A) \approx (2\pi\mu_R)^2 \sigma_R^2 = 4\pi^2 \mu_R^2 \sigma_R^2.$$

Example 2.3.2. Let W be a measured weight with mean μ_W and variance σ_W^2, and suppose density d is fixed. Define

$$V = f(W) = \frac{1}{d} W^{1/3}.$$

Then

$$f'(w) = \frac{1}{3d} w^{-2/3}, \qquad f''(w) = -\frac{2}{9d} w^{-5/3}.$$

Therefore,

$$\mathbb{E}[V] \approx \frac{1}{d} \mu_W^{1/3} - \frac{1}{9d} \mu_W^{-5/3} \sigma_W^2,$$

and

$$\mathrm{Var}(V) \approx \frac{1}{9d^2} \mu_W^{-4/3} \sigma_W^2.$$

2.4 Two-Dimensional Delta Method

Let U and V have means μ_U, μ_V, variances σ_U^2, σ_V^2, and covariance $\sigma_{U,V}$. Let

$$W = f(U, V),$$

where $f : \mathbb{R}^2 \to \mathbb{R}$ is smooth.

Using a second-order Taylor expansion around (μ_U, μ_V),

$$\begin{aligned} f(U,V) \approx\ & f(\mu_U,\mu_V) + f_u(\mu_U,\mu_V)(U-\mu_U) + f_v(\mu_U,\mu_V)(V-\mu_V) \\ & + \frac{1}{2} f_{uu}(\mu_U,\mu_V)(U-\mu_U)^2 + \frac{1}{2} f_{vv}(\mu_U,\mu_V)(V-\mu_V)^2 \\ & + f_{uv}(\mu_U,\mu_V)(U-\mu_U)(V-\mu_V). \end{aligned}$$

Thus

$$\mathbb{E}[f(U,V)] \approx f(\mu_U,\mu_V) + \frac{1}{2} f_{uu}(\mu_U,\mu_V)\sigma_U^2 + \frac{1}{2} f_{vv}(\mu_U,\mu_V)\sigma_V^2 + f_{uv}(\mu_U,\mu_V)\sigma_{U,V}.$$

Using the first-order approximation,

$$\mathrm{Var}(f(U,V)) \approx f_u(\mu_U,\mu_V)^2\sigma_U^2 + f_v(\mu_U,\mu_V)^2\sigma_V^2 + 2 f_u(\mu_U,\mu_V) f_v(\mu_U,\mu_V)\sigma_{U,V}.$$

2.5 Ratio Example

Let

$$f(u,v) = \frac{v}{u}.$$

Then

$$f_u(u,v) = -\frac{v}{u^2}, \qquad f_v(u,v) = \frac{1}{u},$$

and

$$f_{uu}(u,v) = \frac{2v}{u^3}, \qquad f_{vv}(u,v) = 0, \qquad f_{uv}(u,v) = -\frac{1}{u^2}.$$

Therefore,

$$\mathbb{E}\left[\frac{V}{U}\right] \approx \frac{\mu_V}{\mu_U} + \frac{\mu_V}{\mu_U^3}\sigma_U^2 - \frac{1}{\mu_U^2}\sigma_{U,V},$$

and

$$\mathrm{Var}\left(\frac{V}{U}\right) \approx \frac{\mu_V^2}{\mu_U^4}\sigma_U^2 + \frac{1}{\mu_U^2}\sigma_V^2 - \frac{2\mu_V}{\mu_U^3}\sigma_{U,V}.$$

2.6 Functions of Sample Means

Let $X_1, \ldots, X_n \overset{\text{iid}}{\sim} X$ with $\mathbb{E}[X] = \mu_X$ and $\text{Var}(X) = \sigma_X^2 < \infty$. Then

$$\text{Var}(\bar{X}) = \frac{\sigma_X^2}{n}.$$

By Chebyshev's inequality,

$$\mathbb{P}(|\bar{X} - \mu_X| > \varepsilon) \leq \frac{\sigma_X^2}{n\varepsilon^2} \to 0.$$

Thus $\bar{X}$ becomes concentrated near μ_X as $n \to \infty$, making the delta method especially useful for nonlinear functions of estimators.

Example 2.6.1. Let $\bar{Y}$ and $\bar{X}$ be independent sample means with

$$\mathbb{E}[\bar{Y}] = \mu_Y, \qquad \mathbb{E}[\bar{X}] = \mu_X,$$

$$\text{Var}(\bar{Y}) = \frac{\sigma_Y^2}{n}, \qquad \text{Var}(\bar{X}) = \frac{\sigma_X^2}{n}, \qquad \text{Cov}(\bar{X}, \bar{Y}) = 0.$$

For the ratio estimator $\bar{Y}/\bar{X}$,

$$\mathbb{E}\left[\frac{\bar{Y}}{\bar{X}}\right] \approx \frac{\mu_Y}{\mu_X} + \frac{\mu_Y \sigma_X^2}{\mu_X^3 n},$$

and

$$\text{Var}\left(\frac{\bar{Y}}{\bar{X}}\right) \approx \frac{\mu_Y^2 \sigma_X^2}{\mu_X^4 n} + \frac{\sigma_Y^2}{\mu_X^2 n}.$$

Remark 2.6.1. The delta method improves when the input random variables are tightly concentrated. This can happen because the underlying variance is small, or because the input is an estimator such as a sample mean whose variance decreases with n.

Chapter 3

Monte Carlo Methods

3.1 Overview

Monte Carlo methods approximate quantities involving random objects by simulation. Let X be a random object taking values in a space $\mathcal{X}$. Common choices include finite sets, $\mathbb{R}$, $\mathbb{R}^k$, or function spaces such as $C[0,T]$.

The general goal is to compute

$$\mu = \mathbb{E}[g(X)]$$

for some function $g : \mathcal{X} \to \mathbb{R}$.

If $g = \mathbb{1}_A$, then

$$\mathbb{E}[g(X)] = \mathbb{P}(X \in A),$$

so probability estimation is a special case of expectation estimation.

3.2 Monte Carlo Estimation

Let $X^{(1)}, \ldots, X^{(n)}$ be iid samples from the distribution of X. Define

$$\hat{\mu}_n = \frac{1}{n} \sum_{i=1}^{n} g(X^{(i)}).$$

Theorem 3.2.1. *If $\mathbb{E}[|g(X)|] < \infty$, then*

$$\hat{\mu}_n \overset{a.s.}{\to} \mu.$$

Proof. This follows directly from the strong law of large numbers applied to the iid random variables $g(X^{(1)}), \ldots, g(X^{(n)})$. □

Theorem 3.2.2. *If $\sigma^2 = \mathrm{Var}(g(X)) < \infty$, then*

$$\sqrt{n}(\hat{\mu}_n - \mu) \overset{d}{\to} \mathcal{N}(0, \sigma^2).$$

Proof. This follows from the central limit theorem applied to $g(X^{(1)}), \dots, g(X^{(n)})$. □

An approximate large-sample confidence interval is

$$\hat{\mu}_n \pm z_{\alpha/2} \frac{\sigma}{\sqrt{n}},$$

where $z_{\alpha/2} = \Phi^{-1}(1 - \alpha/2)$.

Remark 3.2.1. Monte Carlo accuracy improves at rate $1/\sqrt{n}$. This convergence rate does not directly depend on the dimension of $\mathcal{X}$, which is one reason Monte Carlo methods are useful in high-dimensional problems.

3.3 Random Number Generation

Most Monte Carlo methods begin with pseudo-random draws

$$U \sim \text{Uniform}(0, 1).$$

These are transformed into samples from more complicated distributions using methods such as inversion or acceptance–rejection.

3.4 The Inversion Method

Theorem 3.4.1. *Let $U \sim \text{Uniform}(0, 1)$ and suppose F is a strictly increasing continuous CDF. Then*

$$X = F^{-1}(U)$$

has CDF F.

Proof. For any x,

$$\mathbb{P}(X \le x) = \mathbb{P}(F^{-1}(U) \le x) = \mathbb{P}(U \le F(x)).$$

Since $U \sim \text{Uniform}(0, 1)$,

$$\mathbb{P}(U \le F(x)) = F(x).$$

Thus X has CDF F. □

Example 3.4.1. Let $X \sim \text{Exponential}(\lambda)$, so

$$F_X(x) = 1 - e^{-\lambda x}, \quad x > 0.$$

Set $u = 1 - e^{-\lambda x}$. Solving for x gives

$$F_X^{-1}(u) = -\frac{\log(1 - u)}{\lambda}.$$

Therefore, if $U \sim \text{Uniform}(0, 1)$, then

$$X = -\frac{\log(1 - U)}{\lambda}$$

has an exponential distribution with rate λ.

3.5 Acceptance–Rejection Sampling

Suppose the target density is f and the proposal density is g. Assume there exists $c > 0$ such that

$$f(x) \le cg(x)$$

for all x.

The acceptance–rejection algorithm is:

Algorithm 1 Acceptance–Rejection Sampling

1: **repeat**
2: Generate $X \sim g$
3: Generate $U \sim \text{Uniform}(0, 1)$
4: **until** $U \le \dfrac{f(X)}{cg(X)}$
5: **return** X

Theorem 3.5.1. *The accepted value X has density f.*

Proof. The joint proposal samples points under the envelope $cg(x)$. Conditional on proposing $X = x$, the point is accepted with probability

$$\frac{f(x)}{cg(x)}.$$

Thus the density of an accepted proposal is proportional to

$$g(x)\frac{f(x)}{cg(x)} = \frac{1}{c}f(x).$$

After normalization, the accepted density is $f(x)$. □

Proposition 3.5.1. *The acceptance probability is $1/c$, and the expected number of proposals required for one accepted sample is c.*

Proof. The acceptance probability is

$$\int g(x)\frac{f(x)}{cg(x)}\,\mathrm{d}x = \frac{1}{c}\int f(x)\,\mathrm{d}x = \frac{1}{c}.$$

The number of proposals until acceptance is geometric with success probability $1/c$, so its expectation is c. □

3.6 Example: Sampling a Standard Normal

The standard normal density is

$$f(x) = \frac{1}{\sqrt{2\pi}} e^{-x^2/2}.$$

Use the Laplace proposal

$$g(x) = \frac{1}{2} e^{-|x|}.$$

For $x > 0$,

$$\frac{f(x)}{g(x)} = \frac{2}{\sqrt{2\pi}} e^{-x^2/2+x}.$$

Maximize the exponent:

$$-\frac{x^2}{2} + x = -\frac{1}{2}(x-1)^2 + \frac{1}{2}.$$

The maximum occurs at $x = 1$, giving

$$c = \frac{2}{\sqrt{2\pi}} e^{1/2} = \sqrt{\frac{2e}{\pi}} < 1.32.$$

Therefore the acceptance probability is approximately

$$\frac{1}{c} \approx 0.76.$$

To sample from the Laplace proposal:

1. Generate $Y \sim \text{Exponential}(1)$.
2. Generate $V \sim \text{Uniform}(0, 1)$.
3. Set

$$X = \begin{cases} Y, & V \le 1/2, \\ -Y, & V > 1/2. \end{cases}$$

3.7 Examples of Monte Carlo Problems

Example 3.7.1. For coin tossing, let $X = (X_1, \ldots, X_n) \in \{0, 1\}^n$. Monte Carlo can estimate quantities such as the probability of a run of 15 consecutive heads in 100 flips or the expected longest run length.

Example 3.7.2. For a shuffled deck, let X be a random permutation of 52 cards. Monte Carlo can estimate probabilities such as all four aces appearing in one bridge hand or the expected number of suits represented in a hand.

Example 3.7.3. For a stochastic process $X = \{X_t : t \in [0, T]\}$, Monte Carlo can estimate functionals such as

$$\mathbb{P}(X_t > \tau_U)$$

or

$$\mathbb{E}\left[\int_0^T X_t^2 \, \mathrm{d}t\right].$$

3.8 Summary

Method	Key idea	Notes
Monte Carlo estimation	Approximate $\mathbb{E}[g(X)]$ by sample averages	Accuracy improves at rate $1/\sqrt{n}$
Inversion method	Use $X = F^{-1}(U)$	Exact when the inverse CDF is available
Acceptance–rejection	Propose from g and accept with probability $f(X)/(cg(X))$	Efficiency depends on choosing a small envelope constant c

Chapter 4

Markov Chains

4.1 Sampling Dependent Random Variables

Suppose X and Y have joint probability mass function

$\mathbb{P}(X = x, Y = y)$	$y = 0$	$y = 1$
$x = 0$	$1/12$	$2/12$
$x = 1$	$6/12$	$3/12$

There are two natural ways to sample from this joint distribution.

4.1.1 Sampling Using a Single Variable

Define a random variable Z taking values in $\{0, 1, 2, 3\}$ with

$$\mathbb{P}(Z = 0) = \frac{1}{12}, \quad \mathbb{P}(Z = 1) = \frac{2}{12}, \quad \mathbb{P}(Z = 2) = \frac{6}{12}, \quad \mathbb{P}(Z = 3) = \frac{3}{12}.$$

Define a mapping f by

$$f(0) = (0, 0), \quad f(1) = (0, 1), \quad f(2) = (1, 0), \quad f(3) = (1, 1).$$

Then sampling Z and returning $f(Z)$ produces a draw from the joint distribution of (X, Y).

4.1.2 Conditional Sampling

Alternatively, sample X first from its marginal distribution:

$$\mathbb{P}(X = 0) = \frac{3}{12}, \qquad \mathbb{P}(X = 1) = \frac{9}{12}.$$

Then sample Y conditionally on the observed value of X. If $X = 0$,

$$\mathbb{P}(Y = 0|X = 0) = \frac{1/12}{3/12} = \frac{1}{3}, \qquad \mathbb{P}(Y = 1|X = 0) = \frac{2/12}{3/12} = \frac{2}{3}.$$

If $X = 1$,

$$\mathbb{P}(Y = 0|X = 1) = \frac{6/12}{9/12} = \frac{2}{3}, \qquad \mathbb{P}(Y = 1|X = 1) = \frac{3/12}{9/12} = \frac{1}{3}.$$

Proposition 4.1.1. *For any two discrete random variables X and Y,*

$$\mathbb{P}(X = x, Y = y) = \mathbb{P}(X = x)\mathbb{P}(Y = y|X = x).$$

Therefore, sampling X from its marginal distribution and then sampling Y from its conditional distribution given X produces a draw from the joint distribution.

Proof. The identity follows from the definition of conditional probability:

$$\mathbb{P}(Y = y|X = x) = \frac{\mathbb{P}(X = x, Y = y)}{\mathbb{P}(X = x)}.$$

Multiplying both sides by $\mathbb{P}(X = x)$ gives the result. □

4.2 Sampling Multiple Dependent Variables

For random variables $X_0, \ldots, X_{n-1}$, the joint distribution can be decomposed as

$$\mathbb{P}(X_0 = x_0, \ldots, X_{n-1} = x_{n-1}) = \mathbb{P}(X_0 = x_0) \prod_{k=1}^{n-1} \mathbb{P}(X_k = x_k|X_0 = x_0, \ldots, X_{k-1} = x_{k-1}).$$

This suggests the following sampling procedure:

1. Sample X_0 from its marginal distribution.
2. Sample X_1 given X_0.
3. Sample X_2 given (X_0, X_1).
4. Continue sampling X_k given the previously sampled values.

The difficulty is that the conditional distributions

$$\mathbb{P}(X_k|X_0, \ldots, X_{k-1})$$

can become very complicated as k grows.

4.3 The Markov Assumption

The Markov assumption simplifies this dependence structure by assuming that the future depends on the past only through the present.

Definition 4.3.1. A sequence of random variables $X_0, X_1, \ldots$ satisfies the Markov property if

$$\mathbb{P}(X_{t+1} = j | X_0, \ldots, X_t) = \mathbb{P}(X_{t+1} = j | X_t)$$

for all states j and all times t.

Remark 4.3.1. The Markov property does not say that the variables are independent. It says that conditional on the current state, the earlier history provides no additional information about the next state.

4.4 Markov Chains

Definition 4.4.1. A Markov chain with finite state space

$$S = \{0, 1, \ldots, K-1\}$$

is a sequence $X_0, X_1, \ldots$ such that each $X_t \in S$ and

$$\mathbb{P}(X_{t+1} = j | X_0, \ldots, X_t) = \mathbb{P}(X_{t+1} = j | X_t)$$

for all $i, j \in S$.

4.4.1 Transition Matrices

Definition 4.4.2. The transition matrix at time t is the matrix $Q(t)$ whose entries are

$$q_{ij}(t) = \mathbb{P}(X_{t+1} = j | X_t = i).$$

If $Q(t)$ is the same for all t, then the chain is time-homogeneous, and we write Q instead of $Q(t)$.

Definition 4.4.3. A matrix Q is stochastic if

$$q_{ij} \geq 0$$

for all i, j, and

$$\sum_{j=0}^{K-1} q_{ij} = 1$$

for every row i.

Definition 4.4.4. A stochastic matrix is doubly stochastic if its columns also sum to one:

$$\sum_{i=0}^{K-1} q_{ij} = 1$$

for every column j.

Remark 4.4.1. Every transition matrix is stochastic, but not every transition matrix is doubly stochastic.

4.5 Examples of Markov Chains

Example 4.5.1. If X_t are iid with

$$\mathbb{P}(X_t = j) = p_j,$$

then the transition probabilities do not depend on the current state. The transition matrix is

$$Q = \begin{bmatrix} p_0 & p_1 & \cdots & p_{K-1} \\ p_0 & p_1 & \cdots & p_{K-1} \\ \vdots & \vdots & \ddots & \vdots \\ p_0 & p_1 & \cdots & p_{K-1} \end{bmatrix}.$$

Example 4.5.2. Consider a two-state chain with states $0 = D$ and $1 = R$. Suppose

$$\mathbb{P}(D \to D) = 0.3, \qquad \mathbb{P}(R \to R) = 0.4.$$

Then

$$Q = \begin{bmatrix} 0.3 & 0.7 \\ 0.6 & 0.4 \end{bmatrix}.$$

Example 4.5.3. For a three-state chain with state space $\{0, 1, 2\}$, one possible transition matrix is

$$Q = \begin{bmatrix} 1/6 & 2/6 & 3/6 \\ 1/2 & 0 & 1/2 \\ 1/4 & 1/4 & 1/2 \end{bmatrix}.$$

Example 4.5.4. A random walk on $\{0, 1, 2, 3, 4\}$ with reflecting boundaries can have transition matrix

$$Q = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 \\ 1/2 & 0 & 1/2 & 0 & 0 \\ 0 & 1/2 & 0 & 1/2 & 0 \\ 0 & 0 & 1/2 & 0 & 1/2 \\ 0 & 0 & 0 & 1 & 0 \end{bmatrix}.$$

At the boundaries, the chain is forced to move inward.

Example 4.5.5. A reflecting random walk that allows the boundary states to remain fixed with probability $1/2$ has transition matrix

$$Q = \begin{bmatrix} 1/2 & 1/2 & 0 & 0 & 0 \\ 1/2 & 0 & 1/2 & 0 & 0 \\ 0 & 1/2 & 0 & 1/2 & 0 \\ 0 & 0 & 1/2 & 0 & 1/2 \\ 0 & 0 & 0 & 1/2 & 1/2 \end{bmatrix}.$$

Example 4.5.6. A nonhomogeneous Markov chain may have a transition matrix that changes with time, such as

$$Q(t) = \begin{bmatrix} \frac{1}{3} + \frac{t}{120} & \frac{2}{3} - \frac{t}{120} \\ \frac{1}{2} - \frac{t}{180} & \frac{1}{2} + \frac{t}{180} \end{bmatrix}.$$

For this to define a valid transition matrix, the entries must remain nonnegative for the time values under consideration.

4.6 One-Step Distributions

Let

$$\pi_i(t) = \mathbb{P}(X_t = i).$$

Then

$$\pi_j(t+1) = \sum_i \pi_i(t) q_{ij}.$$

In row-vector notation,

$$\pi(t+1) = \pi(t)Q.$$

Proof. By the law of total probability,

$$\mathbb{P}(X_{t+1} = j) = \sum_i \mathbb{P}(X_{t+1} = j, X_t = i).$$

Using the product rule,

$$\mathbb{P}(X_{t+1} = j, X_t = i) = \mathbb{P}(X_t = i)\mathbb{P}(X_{t+1} = j | X_t = i).$$

Therefore,

$$\pi_j(t+1) = \sum_i \pi_i(t) q_{ij}.$$

□

4.7 Multi-Step Transition Probabilities

For a time-homogeneous Markov chain,

$$\mathbb{P}(X_{t+m} = j | X_t = i) = (Q^m)_{ij}.$$

Thus, if the distribution at time t is $\pi(t)$, then

$$\pi(t+m) = \pi(t)Q^m.$$

For a nonhomogeneous Markov chain,

$$\mathbb{P}(X_{t+m} = j | X_t = i) = \left(Q(t)Q(t+1)\cdots Q(t+m-1)\right)_{ij}.$$

Proposition 4.7.1. *For a time-homogeneous Markov chain,*

$$\mathbb{P}(X_{t+m} = j | X_t = i) = (Q^m)_{ij}.$$

Proof. The case $m = 1$ follows from the definition of Q. For $m = 2$,

$$\mathbb{P}(X_{t+2} = j | X_t = i) = \sum_{\ell} \mathbb{P}(X_{t+2} = j, X_{t+1} = \ell | X_t = i).$$

Using the Markov property,

$$= \sum_{\ell} \mathbb{P}(X_{t+1} = \ell | X_t = i) \mathbb{P}(X_{t+2} = j | X_{t+1} = \ell) = \sum_{\ell} q_{i\ell} q_{\ell j} = (Q^2)_{ij}.$$

Repeating this argument gives the result for general m. □

4.8 Irreducibility

Definition 4.8.1. State j is accessible from state i if there exists $n \geq 1$ such that

$$(Q^n)_{ij} > 0.$$

Definition 4.8.2. A Markov chain is irreducible if every state is accessible from every other state. Equivalently, for all i, j, there exists $n \geq 1$ such that

$$(Q^n)_{ij} > 0.$$

If this condition fails, the chain is reducible.

Example 4.8.1. The transition matrix

$$Q = \begin{bmatrix} 1 & 0 & 0 \\ 0.5 & 0.5 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

is reducible. State 0 is absorbing, state 2 is absorbing, and state 1 can never reach state 2.

Example 4.8.2. The transition matrix

$$Q = \begin{bmatrix} 0.5 & 0.5 & 0 \\ 0.2 & 0.3 & 0.5 \\ 0.1 & 0.6 & 0.3 \end{bmatrix}$$

is irreducible. From state 0, the chain can reach state 1 directly and state 2 through state 1. From state 2, the chain can reach states 0 and 1 with positive probability.

4.9 Periodicity

Definition 4.9.1. The period of state i is

$$d(i) = \gcd\{n \geq 1 : (Q^n)_{ii} > 0\}.$$

If $d(i) = 1$, state i is aperiodic. If $d(i) > 1$, state i is periodic.

Remark 4.9.1. In an irreducible finite Markov chain, all states have the same period. Therefore, one can refer to the period of the chain.

Example 4.9.1. The transition matrix

$$Q = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{bmatrix}$$

is periodic with period 3. The chain moves deterministically as

$$0 \to 1 \to 2 \to 0 \to \cdots .$$

A return to any state can occur only after $3, 6, 9, \ldots$ steps.

Example 4.9.2. The transition matrix

$$Q = \begin{bmatrix} 0.5 & 0.5 & 0 \\ 0.2 & 0.3 & 0.5 \\ 0.3 & 0.3 & 0.4 \end{bmatrix}$$

is aperiodic because each state has a positive self-loop:

$$q_{00} > 0, \qquad q_{11} > 0, \qquad q_{22} > 0.$$

Hence each state can return to itself in one step, so the period is 1.

4.10 Recurrence and Transience

Definition 4.10.1. A state i is recurrent if, starting from i, the probability of eventually returning to i is 1.

Definition 4.10.2. A state i is transient if, starting from i, there is a positive probability of never returning to i.

Theorem 4.10.1. *In a finite irreducible Markov chain, every state is recurrent.*

Remark 4.10.1. Reducible chains may contain transient states. For example, if a state can move into an absorbing class and can never return, then that state is transient.

4.11 Stationary Distributions

Definition 4.11.1. A probability vector π is a stationary distribution for a Markov chain with transition matrix Q if

$$\pi Q = \pi,$$

where

$$\pi_i \geq 0, \qquad \sum_i \pi_i = 1.$$

If $X_t \sim \pi$ and $\pi Q = \pi$, then

$$X_{t+1} \sim \pi.$$

Thus a stationary distribution is unchanged by one step of the Markov chain.

Proposition 4.11.1. *If Q is doubly stochastic on a finite state space with K states, then*

$$\pi = \left(\frac{1}{K}, \ldots, \frac{1}{K}\right)$$

is a stationary distribution.

Proof. For each j,

$$(\pi Q)_j = \sum_{i=0}^{K-1} \frac{1}{K} q_{ij} = \frac{1}{K} \sum_{i=0}^{K-1} q_{ij}.$$

Since Q is doubly stochastic, each column sums to 1, so

$$(\pi Q)_j = \frac{1}{K}.$$

Therefore $\pi Q = \pi$. □

4.12 Ergodicity

Definition 4.12.1. A finite Markov chain is ergodic if it is irreducible and aperiodic.

Theorem 4.12.1. *If a finite Markov chain is irreducible and aperiodic, then it has a unique stationary distribution π, and for any initial distribution $\pi^{(0)}$,*

$$\lim_{t \to \infty} \pi^{(0)} Q^t = \pi.$$

Remark 4.12.1. Irreducibility ensures that the chain has one communicating class, while aperiodicity prevents deterministic cycling. Together, these conditions imply convergence to a unique long-run distribution.

Remark 4.12.2. If irreducibility fails, the limiting behavior may depend on the starting state. If aperiodicity fails, the chain may continue oscillating even if it has a stationary distribution.

4.13 Summary

Property	Meaning
Stochastic matrix	Nonnegative entries and rows summing to one
Doubly stochastic matrix	Rows and columns both sum to one
Irreducible	Every state can reach every other state in some number of steps
Reducible	Some states cannot be reached from others
Periodic	Returns to a state occur only in multiples of some integer $d > 1$
Aperiodic	The greatest common divisor of return times is 1
Recurrent	Starting from the state, the chain returns with probability 1
Transient	Starting from the state, there is positive probability of never returning
Stationary distribution	A probability vector π satisfying $\pi Q = \pi$
Ergodic	Irreducible and aperiodic; guarantees convergence to a unique stationary distribution

Chapter 5

The Metropolis–Hastings Algorithm

5.1 Overview

The Metropolis–Hastings algorithm is a general-purpose method for sampling from a target distribution π when direct sampling is difficult. The key idea is to construct a Markov chain whose stationary distribution is π. After enough iterations, samples from the chain are approximately distributed according to π.

5.2 Self-Avoiding Paths

Definition 5.2.1. A path of length N in $\mathbb{Z}^2$ is defined by a sequence of direction vectors

$$\delta(i) \in \{(-1,0),(1,0),(0,-1),(0,1)\}, \qquad i = 0,\dots,N-1.$$

The associated sequence of points is

$$p(0) = (0,0),$$

and

$$p(k) = \sum_{i=0}^{k-1} \delta(i), \qquad k = 1,\dots,N.$$

The path is the ordered sequence

$$[p(0), p(1), \dots, p(N)].$$

Definition 5.2.2. A path is self-avoiding if all points $p(0), p(1), \dots, p(N)$ are distinct.

A self-avoiding path never visits the same lattice point twice.

5.3 Uniform Sampling of Self-Avoiding Paths

There are 4^N possible direction sequences of length N, but only some of them produce self-avoiding paths. A simple acceptance–rejection sampler is:

Algorithm 2 Acceptance–Rejection for Uniform Self-Avoiding Paths

1: **repeat**
2: Sample $\delta(0), \ldots, \delta(N-1)$ independently and uniformly from $\{(-1,0),(1,0),(0,-1),(0,1)\}$
3: Construct $p(0), \ldots, p(N)$
4: **until** p is self-avoiding
5: **return** p

This algorithm samples uniformly from the set of self-avoiding paths because every direction sequence is proposed with equal probability, and accepted paths are exactly the valid self-avoiding paths.

Remark 5.3.1. For large N, acceptance–rejection becomes inefficient because the probability that a uniformly sampled direction sequence is self-avoiding becomes very small.

5.4 Number of Bends

Definition 5.4.1. The number of bends in a path is

$$b(p) = \left|\{i \in \{0, \ldots, N-2\} : \delta(i+1) \neq \delta(i)\}\right|.$$

This statistic measures how frequently the direction of the path changes.

5.5 Detailed Balance

Let $S = \{1, \ldots, K\}$ be a finite state space, and let π be a probability vector on S. Let $Q = (q_{ij})$ be a Markov transition matrix.

Definition 5.5.1. The transition matrix Q satisfies detailed balance with respect to π if

$$\pi_i q_{ij} = \pi_j q_{ji}$$

for all $i, j \in S$.

Theorem 5.5.1. *If Q satisfies detailed balance with respect to π, then π is a stationary distribution of Q.*

Proof. For any fixed state k,

$$(\pi Q)_k = \sum_i \pi_i q_{ik}.$$

By detailed balance,

$$\pi_i q_{ik} = \pi_k q_{ki}.$$

Therefore,

$$(\pi Q)_k = \sum_i \pi_k q_{ki} = \pi_k \sum_i q_{ki}.$$

Since Q is stochastic, the rows sum to one, so

$$\sum_i q_{ki} = 1.$$

Hence

$$(\pi Q)_k = \pi_k.$$

Because this holds for every k, $\pi Q = \pi$. □

Remark 5.5.1. Detailed balance is a sufficient condition for stationarity. It is not necessary: some chains have stationary distributions without satisfying detailed balance.

5.6 Metropolis–Hastings Construction

Suppose the state space S is viewed as a graph. The vertices are possible states, and N_i denotes the set of neighbors of state i. Let $n_i = |N_i|$.

The target distribution is π, where $\pi_i > 0$ for all states of interest.

5.6.1 Uniform Neighbor Proposal

Suppose that from state i, we propose a neighbor $j \in N_i$ uniformly:

$$r_{ij} = \frac{1}{n_i}.$$

The Metropolis–Hastings acceptance probability is

$$\alpha(i,j) = \min\left(1, \frac{\pi_j r_{ji}}{\pi_i r_{ij}}\right) = \min\left(1, \frac{\pi_j n_i}{\pi_i n_j}\right).$$

Thus,

$$q_{ij} = \frac{1}{n_i} \min\left(1, \frac{\pi_j n_i}{\pi_i n_j}\right), \qquad i \neq j,$$

and

$$q_{ii} = 1 - \sum_{j \neq i} q_{ij}.$$

Theorem 5.6.1. *The Metropolis–Hastings transition matrix satisfies detailed balance with respect to π.*

Proof. For neighboring states i and j,

$$\pi_i q_{ij} = \pi_i \frac{1}{n_i} \min\left(1, \frac{\pi_j n_i}{\pi_i n_j}\right).$$

This can be written as

$$\pi_i q_{ij} = \min\left(\frac{\pi_i}{n_i}, \frac{\pi_j}{n_j}\right).$$

Similarly,

$$\pi_j q_{ji} = \pi_j \frac{1}{n_j} \min\left(1, \frac{\pi_i n_j}{\pi_j n_i}\right) = \min\left(\frac{\pi_j}{n_j}, \frac{\pi_i}{n_i}\right).$$

Therefore,

$$\pi_i q_{ij} = \pi_j q_{ji}.$$

If i and j are not neighbors, then both transition probabilities are zero, so detailed balance also holds. □

5.6.2 Weighted Neighbor Proposal

Alternatively, propose a neighbor $j \in N_i$ with probability

$$r_{ij} = \frac{\pi_j}{\sum_{u \in N_i} \pi_u}.$$

Then the Metropolis–Hastings acceptance probability is

$$\alpha(i, j) = \min\left(1, \frac{\pi_j r_{ji}}{\pi_i r_{ij}}\right).$$

Substituting the proposal probabilities gives

$$\alpha(i, j) = \min\left(1, \frac{\sum_{u \in N_i} \pi_u}{\sum_{v \in N_j} \pi_v}\right).$$

Remark 5.6.1. The Metropolis–Hastings algorithm only requires ratios of target probabilities. If

$$\pi_i = \frac{w_i}{Z},$$

where Z is an unknown normalizing constant, then

$$\frac{\pi_j}{\pi_i} = \frac{w_j/Z}{w_i/Z} = \frac{w_j}{w_i}.$$

Thus Z cancels.

5.7 Metropolis–Hastings for Uniform Self-Avoiding Paths

Let S be the set of all self-avoiding paths of length N. The uniform target distribution is

$$\pi_p = \frac{1}{|S|}, \qquad p \in S.$$

Two self-avoiding paths are neighbors if one can be obtained from the other by changing exactly one direction vector and the resulting path remains self-avoiding.

Since π_p is constant over S, the acceptance probability for a uniform neighbor proposal becomes

$$\alpha(p, p') = \min\left(1, \frac{|N_p|}{|N_{p'}|}\right).$$

The algorithm is:

Algorithm 3 Metropolis–Hastings for Uniform Self-Avoiding Paths

Initialize X_0 as a valid self-avoiding path, for example the straight path with all directions $(1, 0)$
for $t = 0, 1, \ldots, T-1$ **do**
    List the self-avoiding neighbors N_{X_t}
    Propose $p' \sim \text{Uniform}(N_{X_t})$
    Compute $|N_{X_t}|$ and $|N_{p'}|$
    Generate $U \sim \text{Uniform}(0, 1)$
    if $U \leq \min\left(1, \frac{|N_{X_t}|}{|N_{p'}|}\right)$ **then**
        $X_{t+1} = p'$
    else
        $X_{t+1} = X_t$
    end if
end for

Remark 5.7.1. If the proposed state has more neighbors than the current state, then the move is accepted with probability less than one. This correction prevents the chain from over-sampling states that are easier to propose.

5.8 Autocorrelation and Convergence Diagnostics

Metropolis–Hastings samples are dependent. Therefore, consecutive samples may not behave like iid draws from the target distribution.

One simple diagnostic is the lag-one autocorrelation of a summary statistic. For self-avoiding paths, a natural summary is the bend count

$$B_t = b(X_t).$$

Given samples $B_0, \dots, B_{n-1}$ with mean $\bar{B}$, the lag-one autocorrelation is

$$\rho_1 = \frac{\sum_{i=0}^{n-2}(B_i - \bar{B})(B_{i+1} - \bar{B})}{\sum_{i=0}^{n-1}(B_i - \bar{B})^2}.$$

Remark 5.8.1. Large positive autocorrelation indicates that the chain is moving slowly through the state space. In practice, one may need longer runs, better proposal mechanisms, or thinning between retained samples.

5.9 Nonuniform Target Distributions

Metropolis–Hastings is not limited to uniform targets. For example, to favor smoother self-avoiding paths, define

$$\pi_p = Ce^{-\gamma b(p)},$$

where $b(p)$ is the number of bends, $\gamma > 0$ controls the strength of the preference for fewer bends, and C is a normalizing constant.

Since C cancels in ratios,

$$\frac{\pi_{p'}}{\pi_p} = \frac{Ce^{-\gamma b(p')}}{Ce^{-\gamma b(p)}} = e^{-\gamma(b(p')-b(p))}.$$

For a uniform neighbor proposal, the acceptance probability becomes

$$\alpha(p, p') = \min\left(1, e^{-\gamma(b(p')-b(p))}\frac{|N_p|}{|N_{p'}|}\right).$$

Remark 5.9.1. When γ is large, paths with many bends are penalized more strongly. When $\gamma = 0$, the target distribution reduces to the uniform distribution on self-avoiding paths.

5.10 Ergodicity and Convergence

Theorem 5.10.1. *If the Metropolis–Hastings chain is irreducible and aperiodic, and if it satisfies detailed balance with respect to π, then*

$$\lim_{t\to\infty} \mathbb{P}(X_t = i) = \pi_i$$

for every state i, regardless of the initial state.

Remark 5.10.1. Detailed balance ensures that π is stationary. Irreducibility and aperiodicity ensure convergence to that stationary distribution.

5.11 Summary

Concept	Description
Self-avoiding path	A path on $\mathbb{Z}^2$ that never visits the same point twice
Uniform SAP sampling	All self-avoiding paths of fixed length have equal probability
Acceptance–rejection	Simple exact method, but inefficient for large N
Metropolis–Hastings	Constructs a Markov chain with target stationary distribution π
Detailed balance	Condition $\pi_i q_{ij} = \pi_j q_{ji}$ ensuring stationarity
Acceptance probability	Corrects for proposal asymmetry and target probability ratios
Normalization cancellation	Unknown constants in π are unnecessary because only ratios are used
Autocorrelation	Measures dependence between successive Markov chain samples
Weighted SAPs	Use $\pi_p \propto e^{-\gamma b(p)}$ to favor paths with fewer bends

Chapter 6

Brownian Motion

6.1 From Random Walk to Brownian Motion

The standard symmetric random walk on $\mathbb{Z}$ is a Markov chain defined by

$$X_0 = 0,$$

and

$$X_{t+1} = X_t + \delta_{t+1},$$

where

$$\mathbb{P}(\delta_{t+1} = 1) = \mathbb{P}(\delta_{t+1} = -1) = \frac{1}{2}.$$

Brownian motion can be viewed as the continuous-time limit of a rescaled random walk. Let $\Delta > 0$ be a small time step. Define

$$B_{n\Delta} = \sqrt{\Delta} \sum_{i=1}^{n} \delta_i,$$

where δ_i are iid with

$$\mathbb{P}(\delta_i = 1) = \mathbb{P}(\delta_i = -1) = \frac{1}{2}.$$

For times between grid points, define B_t by linear interpolation.

As $\Delta \to 0$, this process converges to standard Brownian motion.

6.2 Definition and Basic Properties

Definition 6.2.1. A standard Brownian motion is a stochastic process $\{B_t : t \geq 0\}$ satisfying:

1. $B_0 = 0$.
2. $B_t - B_s \sim \mathcal{N}(0, t - s)$ for $0 \leq s < t$.

3. For $0 = t_0 < t_1 < \cdots < t_n$, the increments
$$B_{t_1} - B_{t_0}, \ldots, B_{t_n} - B_{t_{n-1}}$$
are independent.

4. The sample paths $t \mapsto B_t$ are continuous almost surely.

6.2.1 Normality of Increments

For $s < t$, choose integers m, n such that
$$s \approx m\Delta, \qquad t \approx n\Delta.$$
Then
$$B_t - B_s \approx \sqrt{\Delta} \sum_{i=m+1}^{n} \delta_i.$$
Rewrite this as
$$B_t - B_s \approx \sqrt{(n-m)\Delta} \left(\frac{1}{\sqrt{n-m}} \sum_{i=m+1}^{n} \delta_i \right).$$
Since each δ_i has mean 0 and variance 1, the central limit theorem gives
$$\frac{1}{\sqrt{n-m}} \sum_{i=m+1}^{n} \delta_i \xrightarrow{d} \mathcal{N}(0,1).$$
Also,
$$(n-m)\Delta \approx t - s.$$
Therefore,
$$B_t - B_s \sim \mathcal{N}(0, t-s)$$
in the limit.

Remark 6.2.1. Independent increments come from the fact that disjoint time intervals use disjoint sets of random walk steps.

6.3 Simulating Brownian Motion

To simulate approximate Brownian motion on $[0, T]$:

Algorithm 4 Approximate Brownian Motion

1: Choose N and set $\Delta = T/N$
2: Set $B_0 = 0$
3: **for** $n = 1, \ldots, N$ **do**
4: Generate $\delta_n \in \{-1, 1\}$ with equal probability
5: Set $B_{n\Delta} = B_{(n-1)\Delta} + \sqrt{\Delta}\delta_n$
6: **end for**
7: Linearly interpolate between grid points

Alternatively, one may generate

$$Z_n \sim \mathcal{N}(0,1)$$

and set

$$B_{n\Delta} = B_{(n-1)\Delta} + \sqrt{\Delta} Z_n.$$

This directly matches the Gaussian increment structure:

$$B_{n\Delta} - B_{(n-1)\Delta} \sim \mathcal{N}(0, \Delta).$$

6.4 Brownian Motion with Drift and Volatility

Definition 6.4.1. Brownian motion with drift μ and volatility $\sigma > 0$ is

$$Y_t = Y_0 + \mu t + \sigma B_t.$$

For $0 \leq s < t$,

$$Y_t - Y_s = \mu(t-s) + \sigma(B_t - B_s).$$

Since

$$B_t - B_s \sim \mathcal{N}(0, t-s),$$

we have

$$Y_t - Y_s \sim \mathcal{N}(\mu(t-s), \sigma^2(t-s)).$$

6.5 Fitting Brownian Motion with Drift and Volatility

Suppose we observe

$$0 = t_0 < t_1 < \cdots < t_n$$

and values

$$Y_{t_0}, Y_{t_1}, \ldots, Y_{t_n}.$$

Define increments

$$\Delta t_i = t_i - t_{i-1}, \qquad \Delta Y_i = Y_{t_i} - Y_{t_{i-1}}.$$

The model implies

$$\Delta Y_i \sim \mathcal{N}(\mu \Delta t_i, \sigma^2 \Delta t_i),$$

independently for $i = 1, \ldots, n$.

The log-likelihood, up to constants, is

$$\ell(\mu, \sigma^2) = -\frac{1}{2} \sum_{i=1}^{n} \left[\log(\sigma^2 \Delta t_i) + \frac{(\Delta Y_i - \mu \Delta t_i)^2}{\sigma^2 \Delta t_i} \right].$$

Proposition 6.5.1. *The maximum likelihood estimator for μ is*

$$\hat{\mu} = \frac{\sum_{i=1}^{n} \Delta Y_i}{\sum_{i=1}^{n} \Delta t_i} = \frac{Y_{t_n} - Y_{t_0}}{t_n - t_0}.$$

Proof. For fixed σ^2, maximizing the log-likelihood in μ is equivalent to minimizing

$$\sum_{i=1}^{n} \frac{(\Delta Y_i - \mu \Delta t_i)^2}{\Delta t_i}.$$

Differentiate with respect to μ:

$$\frac{\partial}{\partial \mu} \sum_{i=1}^{n} \frac{(\Delta Y_i - \mu \Delta t_i)^2}{\Delta t_i} = -2 \sum_{i=1}^{n} (\Delta Y_i - \mu \Delta t_i).$$

Setting this equal to zero gives

$$\sum_{i=1}^{n} \Delta Y_i = \mu \sum_{i=1}^{n} \Delta t_i.$$

Thus,

$$\hat{\mu} = \frac{\sum_{i=1}^{n} \Delta Y_i}{\sum_{i=1}^{n} \Delta t_i}.$$

□

The maximum likelihood estimator for σ^2 is

$$\hat{\sigma}^2 = \frac{1}{n} \sum_{i=1}^{n} \frac{(\Delta Y_i - \hat{\mu} \Delta t_i)^2}{\Delta t_i}.$$

An approximately unbiased version replaces n by $n-1$:

$$\hat{\sigma}^2 = \frac{1}{n-1} \sum_{i=1}^{n} \frac{(\Delta Y_i - \hat{\mu} \Delta t_i)^2}{\Delta t_i}.$$

Exercise 6.5.1. Write a function that takes times

$$(t_0, t_1, \ldots, t_n)$$

and observed values

$$(Y_{t_0}, Y_{t_1}, \ldots, Y_{t_n})$$

and returns

$$Y_0, \qquad \hat{\mu}, \qquad \hat{\sigma}.$$

6.6 Geometric Brownian Motion

Brownian motion with drift can take negative values. To model positive quantities such as stock prices, use geometric Brownian motion.

Definition 6.6.1. A geometric Brownian motion is

$$X_t = X_0 e^{\mu t + \sigma B_t}.$$

Equivalently,

$$\log X_t = \log X_0 + \mu t + \sigma B_t.$$

For $0 \leq s < t$,

$$\log X_t - \log X_s = \mu(t-s) + \sigma(B_t - B_s),$$

so

$$\log X_t - \log X_s \sim \mathcal{N}(\mu(t-s), \sigma^2(t-s)).$$

Therefore,

$$\frac{X_t}{X_s}$$

is lognormal.

Exercise 6.6.1. Fit a geometric Brownian motion model to positive observations

$$X_{t_0}, X_{t_1}, \ldots, X_{t_n}.$$

Hint: apply the Brownian motion with drift estimators to the transformed data

$$Y_{t_i} = \log X_{t_i}.$$

Return

$$X_0, \qquad \hat{\mu}, \qquad \hat{\sigma}.$$

6.7 Higher-Dimensional Brownian Motion

A d-dimensional Brownian motion is a process

$$B_t = (B_t^{(1)}, \ldots, B_t^{(d)})$$

where the coordinate processes are independent one-dimensional Brownian motions.

For two-dimensional Brownian motion,

$$B_t = (B_t^{(1)}, B_t^{(2)}).$$

Then

$$B_t - B_s \sim \mathcal{N}\left(\begin{bmatrix}0\\0\end{bmatrix}, (t-s)I_2\right).$$

A simple two-dimensional random walk approximation uses steps chosen from

$$(dx, dy) \in \{(1,0), (-1,0), (0,1), (0,-1)\}.$$

To match Brownian scaling, the increments should be scaled by $\sqrt{\Delta}$:

$$(X_{n\Delta}, Y_{n\Delta}) = (X_{(n-1)\Delta}, Y_{(n-1)\Delta}) + \sqrt{\Delta}(dx_n, dy_n).$$

Remark 6.7.1. If the four coordinate directions are chosen with probability 1/4, then each coordinate has variance $\Delta/2$ per step, not Δ. To obtain standard two-dimensional Brownian motion with covariance ΔI_2, one may instead generate independent Gaussian increments

$$Z_n^{(1)}, Z_n^{(2)} \overset{\text{iid}}{\sim} \mathcal{N}(0,1)$$

and set

$$B_{n\Delta} = B_{(n-1)\Delta} + \sqrt{\Delta}(Z_n^{(1)}, Z_n^{(2)}).$$

Exercise 6.7.1. Create a function that takes:

1. total time T,
2. starting point (x_0, y_0),
3. number of steps N with $\Delta = T/N$,

and returns an $(N+1) \times 3$ array whose columns are:

$$t, \qquad X_t, \qquad Y_t.$$

6.8 Summary

Property	Description
Origin	Brownian motion is the limit of a scaled symmetric random walk
Increment distribution	$B_t - B_s \sim \mathcal{N}(0, t-s)$
Independent increments	Increments over disjoint time intervals are independent
Continuity	Sample paths are continuous almost surely
Simulation	Use increments $\sqrt{\Delta} Z_i$ with $Z_i \sim \mathcal{N}(0,1)$
Drift and volatility	$Y_t = Y_0 + \mu t + \sigma B_t$
Geometric Brownian motion	$X_t = X_0 e^{\mu t + \sigma B_t}$ ensures positive values
Higher dimensions	Use independent Brownian motions as coordinates

Chapter 7

Stochastic Differential Equations

7.1 Ordinary Differential Equations

An ordinary differential equation has the form

$$y'(t) = f(t, y(t)),$$

with initial condition

$$y(0) = y_0.$$

Equivalently, one may write

$$\mathrm{d}y(t) = f(t, y(t))\,\mathrm{d}t.$$

Integrating from 0 to t gives

$$y(t) = y(0) + \int_0^t f(s, y(s))\,\mathrm{d}s.$$

Example 7.1.1. The ODE

$$y'(t) = y(t), \qquad y(0) = 1$$

has solution

$$y(t) = e^t.$$

7.2 Euler's Method

Euler's method approximates the solution of an ODE by replacing the derivative with a finite difference. If

$$y'(t) = f(t, y(t)),$$

then for small $\Delta > 0$,

$$y(t + \Delta) - y(t) \approx f(t, y(t))\Delta.$$

Thus

$$y(t+\Delta) \approx y(t) + f(t, y(t))\Delta.$$

Starting from $y_0 = y(0)$, the Euler scheme is

$$y_{n+1} = y_n + f(t_n, y_n)\Delta, \qquad t_n = n\Delta.$$

Algorithm 5 Euler's Method

1: Choose step size $\Delta > 0$ and final time T
2: Set $N = T/\Delta$
3: Initialize $y_0 = y(0)$
4: **for** $n = 0, \ldots, N-1$ **do**
5: Set $t_n = n\Delta$
6: Update $y_{n+1} = y_n + f(t_n, y_n)\Delta$
7: **end for**

7.3 A One-Dimensional ODE Example

Consider

$$x'(t) = f(x, t)$$

with

$$f(x,t) = \frac{\sin(2\pi t) + 2\pi t \cos(2\pi t)}{x}.$$

Then

$$\frac{\mathrm{d}x}{\mathrm{d}t} = \frac{\sin(2\pi t) + 2\pi t \cos(2\pi t)}{x}.$$

Multiplying both sides by $x\,\mathrm{d}t$ gives

$$x\,\mathrm{d}x = [\sin(2\pi t) + 2\pi t \cos(2\pi t)]\ \mathrm{d}t.$$

Integrating,

$$\frac{1}{2}x^2 = t\sin(2\pi t) + C.$$

Therefore,

$$x(t) = \sqrt{2t\sin(2\pi t) + 2C}.$$

If $x(0) = 5$, then

$$25 = 2C,$$

so

$$x(t) = \sqrt{2t\sin(2\pi t) + 25}.$$

7.4 Higher-Dimensional ODEs

The Euler scheme extends naturally to systems of ODEs. Let

$$x(t) \in \mathbb{R}^d$$

and suppose

$$\dot{x}(t) = f(x(t), t),$$

where

$$f : \mathbb{R}^d \times [0, \infty) \to \mathbb{R}^d.$$

Starting from $x(0) = x_0$, the Euler update is

$$x_{n+1} = x_n + \Delta f(x_n, t_n), \qquad t_n = n\Delta.$$

7.4.1 A Two-Dimensional Rotating and Contracting System

Let $d = 2$ and consider

$$\dot{x}_1 = -a(t)x_1 - b(t)x_2,$$
$$\dot{x}_2 = b(t)x_1 - a(t)x_2.$$

Write

$$x_1 = r\cos\theta, \qquad x_2 = r\sin\theta.$$

Then

$$\dot{x}_1 = \dot{r}\cos\theta - r\dot{\theta}\sin\theta,$$

and

$$\dot{x}_2 = \dot{r}\sin\theta + r\dot{\theta}\cos\theta.$$

Using the system equations,

$$\dot{x}_1 = -a(t)r\cos\theta - b(t)r\sin\theta,$$

and

$$\dot{x}_2 = b(t)r\cos\theta - a(t)r\sin\theta.$$

To derive the radial equation, compute

$$\dot{r} = \dot{x}_1\cos\theta + \dot{x}_2\sin\theta.$$

Substituting,

$$\dot{r} = [-a(t)r\cos\theta - b(t)r\sin\theta]\cos\theta + [b(t)r\cos\theta - a(t)r\sin\theta]\sin\theta.$$

The $b(t)$ terms cancel, giving

$$\dot{r} = -a(t)r(\cos^2\theta + \sin^2\theta) = -a(t)r.$$

To derive the angular equation, use

$$r\dot{\theta} = -\dot{x}_1 \sin\theta + \dot{x}_2 \cos\theta.$$

Substituting,

$$r\dot{\theta} = -[-a(t)r\cos\theta - b(t)r\sin\theta]\sin\theta + [b(t)r\cos\theta - a(t)r\sin\theta]\cos\theta.$$

The $a(t)$ terms cancel, giving

$$r\dot{\theta} = b(t)r(\sin^2\theta + \cos^2\theta) = b(t)r.$$

Therefore,

$$\dot{\theta} = b(t).$$

Thus the system reduces to

$$\dot{r} = -a(t)r, \qquad \dot{\theta} = b(t).$$

Solving,

$$r(t) = r(0)\exp\left(-\int_0^t a(s)\,\mathrm{d}s\right),$$

and

$$\theta(t) = \theta(0) + \int_0^t b(s)\,\mathrm{d}s.$$

7.5 Brownian Motion Recap

Brownian motion can be viewed as the limit of a rescaled random walk. Let $\Delta > 0$ and let

$$\delta_1, \delta_2, \ldots \overset{\text{iid}}{\sim} \mathcal{N}(0,1).$$

Define

$$B_0 = 0,$$

and

$$B_{n\Delta} = B_{(n-1)\Delta} + \sqrt{\Delta}\delta_n.$$

As $\Delta \to 0$, this converges to Brownian motion.

Definition 7.5.1. A standard Brownian motion $\{B_t : t \geq 0\}$ satisfies:

1. $B_0 = 0$.
2. For $0 = t_0 < t_1 < \cdots < t_k$, the increments

   $$B_{t_1} - B_{t_0}, \ldots, B_{t_k} - B_{t_{k-1}}$$

   are independent.
3. For $t > s$,

   $$B_t - B_s \sim \mathcal{N}(0, t-s).$$

4. The paths $t \mapsto B_t$ are continuous but nowhere differentiable almost surely.

Proposition 7.5.1. *For standard Brownian motion,*

$$\operatorname{Cov}(B_s, B_t) = \min(s, t).$$

Proof. Assume without loss of generality that $s \le t$. Then

$$B_t = B_s + (B_t - B_s).$$

Therefore,

$$\operatorname{Cov}(B_s, B_t) = \operatorname{Cov}(B_s, B_s) + \operatorname{Cov}(B_s, B_t - B_s).$$

By independent increments, B_s and $B_t - B_s$ are independent, so

$$\operatorname{Cov}(B_s, B_t - B_s) = 0.$$

Also,

$$\operatorname{Cov}(B_s, B_s) = \operatorname{Var}(B_s) = s.$$

Thus

$$\operatorname{Cov}(B_s, B_t) = s = \min(s, t).$$

□

7.6 Stochastic Differential Equations

An SDE generalizes an ODE by adding infinitesimal random shocks. A one-dimensional SDE has the form

$$\mathrm{d}S_t = f(t, S_t)\,\mathrm{d}t + g(t, S_t)\,\mathrm{d}B_t.$$

The function f is called the drift coefficient, and g is called the diffusion coefficient.

For small $\Delta > 0$,

$$S_{t+\Delta} - S_t \approx f(t, S_t)\Delta + g(t, S_t)(B_{t+\Delta} - B_t).$$

Since

$$B_{t+\Delta} - B_t \sim \mathcal{N}(0, \Delta),$$

we can write

$$B_{t+\Delta} - B_t = \sqrt{\Delta} Z, \qquad Z \sim \mathcal{N}(0, 1).$$

7.7 Euler–Maruyama Method

The Euler–Maruyama method is the stochastic analogue of Euler's method. Let $t_n = n\Delta$. The update is

$$S_{n+1} = S_n + f(t_n, S_n)\Delta + g(t_n, S_n)\sqrt{\Delta} Z_n,$$

where

$$Z_n \overset{\text{iid}}{\sim} \mathcal{N}(0,1).$$

Algorithm 6 Euler–Maruyama Method

1: Choose step size $\Delta > 0$ and final time T
2: Set $N = T/\Delta$
3: Initialize S_0
4: **for** $n = 0, \dots, N-1$ **do**
5: Generate $Z_n \sim \mathcal{N}(0,1)$
6: Set $t_n = n\Delta$
7: Update

$$S_{n+1} = S_n + f(t_n, S_n)\Delta + g(t_n, S_n)\sqrt{\Delta} Z_n$$

8: **end for**

Remark 7.7.1. One may also approximate Brownian increments using random variables taking values $\pm\sqrt{\Delta}$ with equal probability. Any increment distribution with mean 0 and variance Δ can yield the same Brownian limit under suitable conditions.

7.8 Ito's Lemma

Theorem 7.8.1. *Let S_t satisfy*

$$\mathrm{d}S_t = f(t, S_t)\,\mathrm{d}t + g(t, S_t)\,\mathrm{d}B_t.$$

Let

$$Y_t = h(t, S_t),$$

where h is sufficiently smooth. Then

$$\mathrm{d}h(t, S_t) = \left[h_t(t, S_t) + h_S(t, S_t) f(t, S_t) + \frac{1}{2} h_{SS}(t, S_t) g^2(t, S_t)\right] \mathrm{d}t + h_S(t, S_t) g(t, S_t)\,\mathrm{d}B_t.$$

Proof. Use a second-order Taylor expansion:

$$\mathrm{d}h \approx h_t\,\mathrm{d}t + h_S\,\mathrm{d}S_t + \frac{1}{2} h_{SS}(\,\mathrm{d}S_t)^2.$$

Substitute

$$\mathrm{d}S_t = f\,\mathrm{d}t + g\,\mathrm{d}B_t.$$

Then

$$(\,\mathrm{d}S_t)^2 = (f\,\mathrm{d}t + g\,\mathrm{d}B_t)^2 = f^2(\,\mathrm{d}t)^2 + 2fg\,\mathrm{d}t\,\mathrm{d}B_t + g^2(\,\mathrm{d}B_t)^2.$$

Using the Ito rules

$$(\,\mathrm{d}t)^2 = 0, \qquad \mathrm{d}t\,\mathrm{d}B_t = 0, \qquad (\,\mathrm{d}B_t)^2 = \,\mathrm{d}t,$$

we get

$$(\,\mathrm{d}S_t)^2 = g^2\,\mathrm{d}t.$$

Therefore,

$$\mathrm{d}h = h_t\,\mathrm{d}t + h_S(f\,\mathrm{d}t + g\,\mathrm{d}B_t) + \frac{1}{2}h_{SS}g^2\,\mathrm{d}t.$$

Collecting terms gives

$$\mathrm{d}h = \left[h_t + h_S f + \frac{1}{2}h_{SS}g^2\right]\,\mathrm{d}t + h_S g\,\mathrm{d}B_t.$$

□

Remark 7.8.1. Ito's lemma is the stochastic version of the chain rule. The additional term

$$\frac{1}{2}h_{SS}g^2\,\mathrm{d}t$$

appears because Brownian increments satisfy $(\,\mathrm{d}B_t)^2 = \mathrm{d}t$.

7.9 Brownian Motion with Drift

Let

$$Y_t = \mu t + \sigma B_t.$$

Then

$$Y_t = h(t, B_t), \qquad h(t, b) = \mu t + \sigma b.$$

Since

$$h_t = \mu, \qquad h_b = \sigma, \qquad h_{bb} = 0,$$

Ito's lemma gives

$$\mathrm{d}Y_t = \mu\,\mathrm{d}t + \sigma\,\mathrm{d}B_t.$$

This process is called arithmetic Brownian motion or Brownian motion with drift and volatility.

7.10 Geometric Brownian Motion

Suppose

$$Y_t = \log X_t$$

satisfies

$$\mathrm{d}Y_t = \mu\,\mathrm{d}t + \sigma\,\mathrm{d}B_t.$$

Then

$$X_t = e^{Y_t}.$$

Let $h(y) = e^y$. Then

$$h'(y) = e^y, \qquad h''(y) = e^y.$$

By Ito's lemma,

$$\mathrm{d}X_t = h'(Y_t)\,\mathrm{d}Y_t + \frac{1}{2}h''(Y_t)\sigma^2\,\mathrm{d}t.$$

Thus

$$\mathrm{d}X_t = X_t(\mu\,\mathrm{d}t + \sigma\,\mathrm{d}B_t) + \frac{1}{2}\sigma^2 X_t\,\mathrm{d}t.$$

Therefore,

$$\mathrm{d}X_t = \left(\mu + \frac{1}{2}\sigma^2\right)X_t\,\mathrm{d}t + \sigma X_t\,\mathrm{d}B_t.$$

Remark 7.10.1. If $\log X_t$ has drift μ, then X_t has drift coefficient $\mu + \sigma^2/2$ in its SDE.

7.11 Log Transform of Geometric Brownian Motion

Suppose instead that X_t satisfies

$$\mathrm{d}X_t = \mu X_t\,\mathrm{d}t + \sigma X_t\,\mathrm{d}B_t.$$

Let

$$Y_t = \log X_t.$$

Then

$$h(x) = \log x, \qquad h'(x) = \frac{1}{x}, \qquad h''(x) = -\frac{1}{x^2}.$$

Using Ito's lemma,

$$\mathrm{d}Y_t = \frac{1}{X_t}\,\mathrm{d}X_t + \frac{1}{2}\left(-\frac{1}{X_t^2}\right)(\sigma^2 X_t^2)\,\mathrm{d}t.$$

Substituting $\mathrm{d}X_t$,

$$\mathrm{d}Y_t = \frac{1}{X_t}(\mu X_t\,\mathrm{d}t + \sigma X_t\,\mathrm{d}B_t) - \frac{1}{2}\sigma^2\,\mathrm{d}t.$$

Thus

$$\mathrm{d}\log X_t = \left(\mu - \frac{1}{2}\sigma^2\right)\,\mathrm{d}t + \sigma\,\mathrm{d}B_t.$$

Exercise 7.11.1. If

$$\mathrm{d}X_t = \mu X_t\,\mathrm{d}t + \sigma X_t\,\mathrm{d}B_t,$$

derive the SDE satisfied by $\log X_t$.

7.12 Calibration of Geometric Brownian Motion

Suppose we observe positive values

$$G_{t_0}, G_{t_1}, \ldots, G_{t_N}$$

at times

$$0 = t_0 < t_1 < \cdots < t_N.$$

Define

$$\Delta t_i = t_{i+1} - t_i,$$

and

$$\Delta \log G_i = \log G_{t_{i+1}} - \log G_{t_i}, \qquad i = 0, \ldots, N-1.$$

If

$$\log G_t = \log G_0 + \mu t + \sigma B_t,$$

then

$$\Delta \log G_i \sim \mathcal{N}(\mu \Delta t_i, \sigma^2 \Delta t_i).$$

The log-likelihood is

$$\log L(\mu, \sigma) = -\frac{N}{2}\log(2\pi) - N \log \sigma - \frac{1}{2}\sum_{i=0}^{N-1} \log(\Delta t_i) - \frac{1}{2\sigma^2}\sum_{i=0}^{N-1} \frac{(\Delta \log G_i - \mu \Delta t_i)^2}{\Delta t_i}.$$

Proposition 7.12.1. *The maximum likelihood estimator for the log-drift parameter μ is*

$$\hat{\mu} = \frac{\sum_{i=0}^{N-1} \Delta \log G_i}{\sum_{i=0}^{N-1} \Delta t_i} = \frac{\log G_{t_N} - \log G_{t_0}}{t_N - t_0}.$$

If $t_0 = 0$, this becomes

$$\hat{\mu} = \frac{\log G_{t_N} - \log G_0}{t_N}.$$

Proof. For fixed σ, maximizing the log-likelihood in μ is equivalent to minimizing

$$\sum_{i=0}^{N-1} \frac{(\Delta \log G_i - \mu \Delta t_i)^2}{\Delta t_i}.$$

Differentiate with respect to μ:

$$-2\sum_{i=0}^{N-1} (\Delta \log G_i - \mu \Delta t_i).$$

Setting this equal to zero gives

$$\sum_{i=0}^{N-1} \Delta \log G_i = \mu \sum_{i=0}^{N-1} \Delta t_i.$$

Therefore,

$$\hat{\mu} = \frac{\sum_{i=0}^{N-1} \Delta \log G_i}{\sum_{i=0}^{N-1} \Delta t_i}.$$

□

Since

$$\Delta \log G_i = \mu \Delta t_i + \sigma \sqrt{\Delta t_i} Z_i,$$

where $Z_i \overset{\text{iid}}{\sim} \mathcal{N}(0,1)$,

$$\hat{\mu} = \mu + \frac{\sigma \sum_{i=0}^{N-1} \sqrt{\Delta t_i} Z_i}{\sum_{i=0}^{N-1} \Delta t_i}.$$

Thus

$$\hat{\mu} \sim \mathcal{N}\left(\mu, \frac{\sigma^2}{t_N - t_0}\right).$$

The maximum likelihood estimator for σ is

$$\hat{\sigma} = \sqrt{\frac{1}{N} \sum_{i=0}^{N-1} \frac{(\Delta \log G_i - \hat{\mu} \Delta t_i)^2}{\Delta t_i}}.$$

An approximately unbiased estimator replaces N with $N-1$:

$$\hat{\sigma} = \sqrt{\frac{1}{N-1} \sum_{i=0}^{N-1} \frac{(\Delta \log G_i - \hat{\mu} \Delta t_i)^2}{\Delta t_i}}.$$

Remark 7.12.1. There are two common parameterizations of geometric Brownian motion:

$$X_t = X_0 e^{\mu t + \sigma B_t},$$

which makes μ the drift of $\log X_t$, and

$$\mathrm{d}X_t = \alpha X_t \,\mathrm{d}t + \sigma X_t \,\mathrm{d}B_t,$$

which makes α the drift coefficient of X_t. They are related by

$$\mu = \alpha - \frac{1}{2}\sigma^2, \qquad \alpha = \mu + \frac{1}{2}\sigma^2.$$

7.13 Summary

Concept	Description
ODE	Deterministic differential equation $\mathrm{d}y = f(t,y)\,\mathrm{d}t$
Euler method	Approximation $y_{n+1} = y_n + f(t_n, y_n)\Delta$
Brownian motion	Continuous stochastic process with independent Gaussian increments
SDE	Differential equation with drift term and Brownian noise term
Euler–Maruyama	Numerical scheme for simulating SDE sample paths
Ito's lemma	Stochastic chain rule with correction term $\frac{1}{2} h_{SS} g^2 \,\mathrm{d}t$

Arithmetic Brownian motion	$\mathrm{d}Y_t = \mu\,\mathrm{d}t + \sigma\,\mathrm{d}B_t$
Geometric Brownian motion	Positive process satisfying $\mathrm{d}X_t = \alpha X_t\,\mathrm{d}t + \sigma X_t\,\mathrm{d}B_t$
GBM calibration	Estimate parameters using Gaussian log-increments

Chapter 8

Directional Derivatives

8.1 Definition

Let $f : \mathbb{R}^2 \to \mathbb{R}$ be differentiable at (x, y), and let

$$\mathbf{u} = (\cos\theta, \sin\theta)$$

be a unit vector. The directional derivative of f in the direction $\mathbf{u}$ is

$$D_{\mathbf{u}} f(x, y) = f_x(x, y)\cos\theta + f_y(x, y)\sin\theta.$$

Equivalently,

$$D_{\mathbf{u}} f(x, y) = \nabla f(x, y)^\top \mathbf{u}.$$

8.2 A Direction-Dependent Example

Consider

$$f(x, y) = \frac{yx^2}{x^2 + y^2},$$

for $(x, y) \neq (0, 0)$, and define

$$f(0, 0) = 0.$$

We study the behavior of f near the origin.

8.2.1 Partial Derivative with Respect to x

For $(x, y) \neq (0, 0)$,

$$f_x(x, y) = \frac{\partial}{\partial x}\left(\frac{yx^2}{x^2 + y^2}\right).$$

Using the quotient rule,

$$f_x(x,y) = \frac{2xy(x^2+y^2) - yx^2(2x)}{(x^2+y^2)^2}.$$

Simplifying,

$$f_x(x,y) = \frac{2xy^3}{(x^2+y^2)^2}.$$

Now approach the origin along the line

$$x = u\cos\theta, \qquad y = u\sin\theta.$$

Then

$$f_x(u\cos\theta, u\sin\theta) = \frac{2(u\cos\theta)(u\sin\theta)^3}{\left((u\cos\theta)^2 + (u\sin\theta)^2\right)^2}.$$

Since

$$(u\cos\theta)^2 + (u\sin\theta)^2 = u^2,$$

we get

$$f_x(u\cos\theta, u\sin\theta) = 2\cos\theta\sin^3\theta.$$

This depends on the direction θ but not on the distance u from the origin.

8.2.2 Partial Derivative with Respect to y

For $(x,y) \neq (0,0)$,

$$f_y(x,y) = \frac{\partial}{\partial y}\left(\frac{yx^2}{x^2+y^2}\right).$$

Using the quotient rule,

$$f_y(x,y) = \frac{x^2(x^2+y^2) - yx^2(2y)}{(x^2+y^2)^2}.$$

Simplifying,

$$f_y(x,y) = \frac{x^2(x^2-y^2)}{(x^2+y^2)^2}.$$

Along the line

$$x = u\cos\theta, \qquad y = u\sin\theta,$$

we obtain

$$f_y(u\cos\theta, u\sin\theta) = \frac{u^2\cos^2\theta\,(u^2\cos^2\theta - u^2\sin^2\theta)}{u^4}.$$

Therefore,

$$f_y(u\cos\theta, u\sin\theta) = \cos^2\theta(\cos^2\theta - \sin^2\theta) = \cos^2\theta\cos(2\theta).$$

Again, the expression depends on the direction of approach.

8.3 Continuity and Differentiability

At the origin,

$$f_x(0,0) = \lim_{h\to 0} \frac{f(h,0) - f(0,0)}{h}.$$

Since

$$f(h,0) = 0,$$

we get

$$f_x(0,0) = 0.$$

Similarly,

$$f_y(0,0) = \lim_{h\to 0} \frac{f(0,h) - f(0,0)}{h}.$$

Since

$$f(0,h) = 0,$$

we get

$$f_y(0,0) = 0.$$

Thus both partial derivatives exist at the origin.

However, f_x is not continuous at the origin because

$$f_x(u\cos\theta, u\sin\theta) = 2\cos\theta\sin^3\theta$$

has different limits for different θ. Similarly,

$$f_y(u\cos\theta, u\sin\theta) = \cos^2\theta\cos(2\theta)$$

is direction-dependent.

Proposition 8.3.1. *The function*

$$f(x,y) = \frac{yx^2}{x^2+y^2}, \qquad f(0,0) = 0,$$

has partial derivatives at the origin, but its partial derivatives are not continuous at the origin.

Remark 8.3.1. The existence of partial derivatives at a point does not imply differentiability at that point. Differentiability is a stronger condition requiring a good linear approximation to the function near the point.

8.4 Directional Derivatives at the Origin

For a unit vector

$$\mathbf{u} = (a, b),$$

the directional derivative at the origin is

$$D_{\mathbf{u}} f(0,0) = \lim_{h \to 0} \frac{f(ha, hb) - f(0,0)}{h}.$$

Now

$$f(ha, hb) = \frac{(hb)(ha)^2}{(ha)^2 + (hb)^2}.$$

Since $a^2 + b^2 = 1$,

$$f(ha, hb) = hba^2.$$

Therefore,

$$D_{\mathbf{u}} f(0,0) = ba^2.$$

If f were differentiable at the origin, then

$$D_{\mathbf{u}} f(0,0) = \nabla f(0,0)^\top \mathbf{u}.$$

But

$$\nabla f(0,0) = (0,0),$$

so differentiability would imply

$$D_{\mathbf{u}} f(0,0) = 0$$

for every direction $\mathbf{u}$. This is false, since for example with

$$\mathbf{u} = \left(\frac{1}{\sqrt{2}}, \frac{1}{\sqrt{2}} \right),$$

we get

$$D_{\mathbf{u}} f(0,0) = \frac{1}{\sqrt{2}} \left(\frac{1}{2} \right) = \frac{1}{2\sqrt{2}}.$$

Proposition 8.4.1. *The function f is not differentiable at $(0,0)$.*

Proof. If f were differentiable at $(0,0)$, then all directional derivatives would be determined by the gradient:

$$D_{\mathbf{u}} f(0,0) = \nabla f(0,0)^\top \mathbf{u}.$$

Since $f_x(0,0) = f_y(0,0) = 0$, this would imply

$$D_{\mathbf{u}} f(0,0) = 0$$

for all unit vectors $\mathbf{u}$. But for

$$\mathbf{u} = \left(\frac{1}{\sqrt{2}}, \frac{1}{\sqrt{2}} \right),$$

we found

$$D_{\mathbf{u}}f(0,0) = \frac{1}{2\sqrt{2}} \neq 0.$$

Therefore, f is not differentiable at the origin. □

Remark 8.4.1. Direction-dependence near a point is often a warning sign that differentiability may fail. Partial derivatives can exist without producing a valid linear approximation.

Chapter 9

Mean-Reverting Processes

9.1 Motivation

Geometric Brownian motion is often used for equity prices, but it is not always appropriate for quantities that fluctuate around a long-run average. Commodity prices, interest rates, and certain physical measurements often exhibit mean reversion.

A standard mean-reverting process is the Ornstein–Uhlenbeck process:

$$\mathrm{d}X_t = \theta(\mu - X_t)\,\mathrm{d}t + \sigma\,\mathrm{d}B_t,$$

where:

- $\theta > 0$ is the speed of mean reversion,
- μ is the long-run mean,
- $\sigma > 0$ is the volatility,
- B_t is standard Brownian motion.

When $X_t > \mu$, the drift

$$\theta(\mu - X_t)$$

is negative, pulling the process downward. When $X_t < \mu$, the drift is positive, pulling the process upward.

9.2 Euler–Maruyama Simulation

To simulate

$$\mathrm{d}X_t = \theta(\mu - X_t)\,\mathrm{d}t + \sigma\,\mathrm{d}B_t$$

on a grid

$$s,\ s+\Delta,\ s+2\Delta, \ldots,$$

the Euler–Maruyama update is

$$X_{s+(j+1)\Delta} = X_{s+j\Delta} + \theta(\mu - X_{s+j\Delta})\Delta + \sigma\sqrt{\Delta}Z_j,$$

where

$$Z_j \overset{\text{iid}}{\sim} \mathcal{N}(0,1).$$

Algorithm 7 Euler–Maruyama Simulation of a Mean-Reverting Process

1: Choose parameters θ, μ, σ, step size Δ, and initial value X_0
2: **for** $j = 0, \ldots, N-1$ **do**
3: Generate $Z_j \sim \mathcal{N}(0,1)$
4: Set
$$X_{(j+1)\Delta} = X_{j\Delta} + \theta(\mu - X_{j\Delta})\Delta + \sigma\sqrt{\Delta}Z_j$$
5: **end for**

9.3 Exact Conditional Distribution

The mean-reverting SDE admits an exact solution. Starting from

$$\mathrm{d}X_t = \theta(\mu - X_t)\,\mathrm{d}t + \sigma\,\mathrm{d}B_t,$$

rewrite as

$$\mathrm{d}X_t + \theta X_t\,\mathrm{d}t = \theta\mu\,\mathrm{d}t + \sigma\,\mathrm{d}B_t.$$

Multiply by the integrating factor $e^{\theta t}$. Define

$$Y_t = e^{\theta t}X_t.$$

Using Ito's product rule,

$$\mathrm{d}Y_t = \theta e^{\theta t}X_t\,\mathrm{d}t + e^{\theta t}\,\mathrm{d}X_t.$$

Substitute the SDE for $\mathrm{d}X_t$:

$$\mathrm{d}Y_t = \theta e^{\theta t}X_t\,\mathrm{d}t + e^{\theta t}\left[\theta(\mu - X_t)\,\mathrm{d}t + \sigma\,\mathrm{d}B_t\right].$$

The X_t terms cancel:

$$\mathrm{d}Y_t = \theta\mu e^{\theta t}\,\mathrm{d}t + \sigma e^{\theta t}\,\mathrm{d}B_t.$$

Integrating from s to t gives

$$e^{\theta t}X_t - e^{\theta s}X_s = \theta\mu\int_s^t e^{\theta u}\,\mathrm{d}u + \sigma\int_s^t e^{\theta u}\,\mathrm{d}B_u.$$

Since

$$\theta\mu\int_s^t e^{\theta u}\,\mathrm{d}u = \mu(e^{\theta t} - e^{\theta s}),$$

we obtain

$$X_t = e^{-\theta(t-s)}X_s + \mu(1 - e^{-\theta(t-s)}) + \sigma e^{-\theta t}\int_s^t e^{\theta u}\,\mathrm{d}B_u.$$

The stochastic integral has mean zero. By Ito isometry,

$$\mathrm{Var}\left(\sigma e^{-\theta t}\int_s^t e^{\theta u}\,\mathrm{d}B_u\right) = \sigma^2 e^{-2\theta t}\int_s^t e^{2\theta u}\,\mathrm{d}u.$$

Thus

$$\mathrm{Var}\left(\sigma e^{-\theta t}\int_s^t e^{\theta u}\,\mathrm{d}B_u\right) = \sigma^2 e^{-2\theta t}\frac{e^{2\theta t} - e^{2\theta s}}{2\theta}.$$

Simplifying,

$$\mathrm{Var}\left(\sigma e^{-\theta t}\int_s^t e^{\theta u}\,\mathrm{d}B_u\right) = \sigma^2\frac{1 - e^{-2\theta(t-s)}}{2\theta}.$$

Theorem 9.3.1. *For $t > s$,*

$$X_t|X_s \sim \mathcal{N}(m_{s,t}, v_{s,t}),$$

where

$$m_{s,t} = e^{-\theta(t-s)}X_s + \mu(1 - e^{-\theta(t-s)}),$$

and

$$v_{s,t} = \sigma^2\frac{1 - e^{-2\theta(t-s)}}{2\theta}.$$

9.4 Exact Simulation on an Arbitrary Time Grid

Let

$$0 = t_0 < t_1 < \cdots < t_N$$

be a time grid. Define

$$\Delta t_i = t_i - t_{i-1}.$$

Then exact simulation uses

$$X_{t_i}|X_{t_{i-1}} \sim \mathcal{N}(m_i, v_i),$$

where

$$m_i = e^{-\theta\Delta t_i}X_{t_{i-1}} + \mu(1 - e^{-\theta\Delta t_i}),$$

and

$$v_i = \sigma^2\frac{1 - e^{-2\theta\Delta t_i}}{2\theta}.$$

Equivalently,

$$X_{t_i} = m_i + \sqrt{v_i}Z_i, \qquad Z_i \overset{\text{iid}}{\sim} \mathcal{N}(0,1).$$

9.5 Long-Run Behavior

From the conditional mean,

$$\mathbb{E}[X_t|X_0] = e^{-\theta t}X_0 + \mu(1 - e^{-\theta t}).$$

Therefore,

$$\lim_{t\to\infty} \mathbb{E}[X_t|X_0] = \mu.$$

From the conditional variance,

$$\mathrm{Var}(X_t|X_0) = \sigma^2 \frac{1 - e^{-2\theta t}}{2\theta}.$$

Therefore,

$$\lim_{t\to\infty} \mathrm{Var}(X_t|X_0) = \frac{\sigma^2}{2\theta}.$$

Thus the limiting stationary distribution is

$$X_\infty \sim \mathcal{N}\left(\mu, \frac{\sigma^2}{2\theta}\right).$$

9.6 Covariance Structure

Assume X_0 is fixed. For $0 < s < t$, the covariance is

$$\mathrm{Cov}(X_s, X_t) = \frac{\sigma^2}{2\theta}\left(e^{-\theta(t-s)} - e^{-\theta(t+s)}\right).$$

Proof. Using the exact solution from time 0,

$$X_t = e^{-\theta t}X_0 + \mu(1 - e^{-\theta t}) + \sigma e^{-\theta t}\int_0^t e^{\theta u}\,\mathrm{d}B_u.$$

Only the stochastic integral contributes to covariance. Thus

$$\mathrm{Cov}(X_s, X_t) = \sigma^2 e^{-\theta(s+t)}\mathrm{Cov}\left(\int_0^s e^{\theta u}\,\mathrm{d}B_u, \int_0^t e^{\theta v}\,\mathrm{d}B_v\right).$$

Since $s < t$, the shared stochastic variation is over $[0, s]$, so by Ito isometry,

$$\mathrm{Cov}(X_s, X_t) = \sigma^2 e^{-\theta(s+t)}\int_0^s e^{2\theta u}\,\mathrm{d}u.$$

Therefore,

$$\mathrm{Cov}(X_s, X_t) = \sigma^2 e^{-\theta(s+t)}\frac{e^{2\theta s} - 1}{2\theta}.$$

Simplifying,

$$\mathrm{Cov}(X_s, X_t) = \frac{\sigma^2}{2\theta}\left(e^{-\theta(t-s)} - e^{-\theta(t+s)}\right).$$

□

Remark 9.6.1. If the process is initialized from its stationary distribution, then

$$\mathrm{Cov}(X_s, X_t) = \frac{\sigma^2}{2\theta} e^{-\theta|t-s|}.$$

9.7 Likelihood for Observed Data

Suppose we observe

$$X_{t_0}, X_{t_1}, \ldots, X_{t_N}$$

at times

$$t_0 < t_1 < \cdots < t_N.$$

Because the process is Markov,

$$f(x_{t_0}, \ldots, x_{t_N}) = f(x_{t_0}) \prod_{i=1}^{N} f(x_{t_i} | x_{t_{i-1}}).$$

Ignoring the initial density term, the conditional log-likelihood is

$$\ell(\mu, \sigma, \theta) = \sum_{i=1}^{N} \left[-\frac{1}{2} \log(2\pi v_i) - \frac{(X_{t_i} - m_i)^2}{2v_i} \right],$$

where

$$m_i = e^{-\theta \Delta t_i} X_{t_{i-1}} + \mu(1 - e^{-\theta \Delta t_i}),$$

and

$$v_i = \sigma^2 \frac{1 - e^{-2\theta \Delta t_i}}{2\theta}.$$

9.8 Simplifying Notation

Define

$$w_i = e^{-\theta \Delta t_i}.$$

Then

$$m_i = w_i X_{t_{i-1}} + \mu(1 - w_i),$$

and

$$v_i = \sigma^2 \frac{1 - w_i^2}{2\theta}.$$

9.9 Maximum Likelihood Estimation

For fixed θ, the transition model can be written as a heteroskedastic Gaussian regression:

$$X_{t_i} - w_i X_{t_{i-1}} = \mu(1 - w_i) + \varepsilon_i,$$

where

$$\varepsilon_i \sim \mathcal{N}\left(0, \sigma^2 \frac{1 - w_i^2}{2\theta}\right).$$

9.9.1 Estimating μ for Fixed θ

For fixed θ, maximizing the likelihood with respect to μ is equivalent to minimizing

$$\sum_{i=1}^{N} \frac{\left[X_{t_i} - w_i X_{t_{i-1}} - \mu(1-w_i)\right]^2}{1-w_i^2}.$$

Since

$$1 - w_i^2 = (1-w_i)(1+w_i),$$

the first-order condition gives

$$\hat{\mu}(\theta) = \frac{\sum_{i=1}^{N} \frac{X_{t_i} - w_i X_{t_{i-1}}}{1+w_i}}{\sum_{i=1}^{N} \frac{1-w_i}{1+w_i}}.$$

9.9.2 Estimating σ for Fixed θ and μ

For fixed θ and μ, define

$$\hat{m}_i = w_i X_{t_{i-1}} + \mu(1-w_i).$$

The MLE for σ^2 is

$$\hat{\sigma}^2(\theta) = \frac{2\theta}{N} \sum_{i=1}^{N} \frac{(X_{t_i} - \hat{m}_i)^2}{1-w_i^2}.$$

Thus

$$\hat{\sigma}(\theta) = \sqrt{\frac{2\theta}{N} \sum_{i=1}^{N} \frac{(X_{t_i} - \hat{m}_i)^2}{1-w_i^2}}.$$

9.9.3 Grid Search over θ

A practical estimation strategy is:

1. Choose a grid of candidate values for $\theta > 0$.
2. For each θ, compute $\hat{\mu}(\theta)$.
3. Compute $\hat{\sigma}(\theta)$.
4. Evaluate the log-likelihood
$$\ell(\hat{\mu}(\theta), \hat{\sigma}(\theta), \theta).$$
5. Choose the θ value that maximizes the profile log-likelihood.

The final estimates are

$$\hat{\theta}, \qquad \hat{\mu}(\hat{\theta}), \qquad \hat{\sigma}(\hat{\theta}).$$

9.10 Parametric Bootstrap Confidence Intervals

A parametric bootstrap can be used to quantify uncertainty in

$$(\theta, \mu, \sigma).$$

Algorithm 8 Parametric Bootstrap for Mean-Reverting Process

1: Fit the model to obtain estimates $(\hat{\theta}, \hat{\mu}, \hat{\sigma})$
2: **for** $j = 1, \ldots, M$ **do**
3: Simulate a dataset from the mean-reverting process using $(\hat{\theta}, \hat{\mu}, \hat{\sigma})$
4: Refit the model to the simulated dataset
5: Store $(\hat{\theta}^{(j)}, \hat{\mu}^{(j)}, \hat{\sigma}^{(j)})$
6: **end for**
7: Use the empirical 2.5% and 97.5% quantiles of the bootstrap estimates as approximate 95% confidence intervals

For example,

$$\left(\hat{\theta}_{0.025}, \hat{\theta}_{0.975}\right)$$

is an approximate 95% bootstrap confidence interval for θ.

9.11 Summary

Concept	Description
Mean reversion	Drift pulls the process toward a long-run mean μ
Ornstein–Uhlenbeck SDE	$\mathrm{d}X_t = \theta(\mu - X_t)\,\mathrm{d}t + \sigma\,\mathrm{d}B_t$
Speed parameter	Larger θ means faster reversion to μ
Euler–Maruyama	Approximate simulation using normal increments
Exact transition	$X_t \mid X_s$ is normally distributed
Stationary distribution	$\mathcal{N}(\mu, \sigma^2/(2\theta))$
Likelihood	Product of Gaussian transition densities
Estimation	Profile over θ, with closed-form $\hat{\mu}(\theta)$ and $\hat{\sigma}(\theta)$
Parametric bootstrap	Simulate and refit to approximate parameter uncertainty

Chapter 10

Boosting Methods

10.1 Empirical Risk Minimization

Suppose we observe training data

$$(x^{(i)}, Y^{(i)}), \qquad i = 1, \ldots, N,$$

where

$$x^{(i)} = (x_1^{(i)}, \ldots, x_k^{(i)})$$

is a predictor vector and $Y^{(i)}$ is the response.

The goal is to choose a prediction function f that minimizes the expected loss

$$\mathbb{E}[\ell(Y, f(X))].$$

Since the population distribution is unknown, we instead minimize the empirical risk

$$R_N(f) = \frac{1}{N} \sum_{i=1}^{N} \ell(Y^{(i)}, f(x^{(i)})).$$

10.2 Examples of Loss Functions

Example 10.2.1. For regression with squared error loss,

$$\ell(y, f(x)) = (y - f(x))^2.$$

The empirical risk is

$$R_N(f) = \frac{1}{N} \sum_{i=1}^{N} (Y^{(i)} - f(x^{(i)}))^2.$$

Example 10.2.2. For binary classification with $Y \in \{0, 1\}$ and predicted probability

$$p(x) = \mathbb{P}(Y = 1 | X = x),$$

the negative log-likelihood loss is

$$\ell(y, p(x)) = -y \log p(x) - (1 - y) \log(1 - p(x)).$$

Equivalently, the empirical risk is

$$-\frac{1}{N} \sum_{i=1}^{N} \left[Y^{(i)} \log p(x^{(i)}) + (1 - Y^{(i)}) \log(1 - p(x^{(i)})) \right].$$

Example 10.2.3. For multiclass classification with $Y \in \{1, \dots, K\}$ and predicted probabilities

$$f(x) = (f_1(x), \dots, f_K(x)),$$

where

$$f_j(x) \approx \mathbb{P}(Y = j | X = x),$$

the cross-entropy loss is

$$\ell(y, f(x)) = -\sum_{j=1}^{K} \mathbb{1}_{\{y=j\}} \log f_j(x).$$

The empirical risk is

$$-\frac{1}{N} \sum_{i=1}^{N} \sum_{j=1}^{K} \mathbb{1}_{\{Y^{(i)}=j\}} \log f_j(x^{(i)}).$$

10.3 Weak Learners and Regularity

Boosting constructs a strong predictor by combining many weak predictors. Let $\mathcal{H}$ denote a class of simple prediction functions. A common choice is the class of decision stumps, which are depth-one trees.

Specifying a regression stump requires:

1. a variable to split on,
2. a threshold,
3. a prediction value at each leaf.

Remark 10.3.1. Without restrictions on f, empirical risk minimization can overfit. For example, under squared error loss, if all $x^{(i)}$ are distinct, one can force

$$f(x^{(i)}) = Y^{(i)}$$

for all i, making the training loss zero. Boosting controls complexity by building f as a sum of simple weak learners, often with small step sizes.

10.4 Gradient Boosting

Suppose we currently have a predictor f. We want to improve it by adding a small multiple of a weak learner:

$$f_{\text{new}} = f + ch,$$

where

$$h \in \mathcal{H}$$

and $c \in \mathbb{R}$ is a step size.

The empirical risk after the update is

$$R_N(f + ch) = \frac{1}{N} \sum_{i=1}^{N} \ell(Y^{(i)}, f(x^{(i)}) + ch(x^{(i)})).$$

For small c, use a first-order Taylor expansion in the second argument:

$$\ell(Y^{(i)}, f(x^{(i)}) + ch(x^{(i)})) \approx \ell(Y^{(i)}, f(x^{(i)})) + c\frac{\partial \ell}{\partial f}(Y^{(i)}, f(x^{(i)}))h(x^{(i)}).$$

Therefore,

$$R_N(f + ch) \approx R_N(f) + \frac{c}{N} \sum_{i=1}^{N} \frac{\partial \ell}{\partial f}(Y^{(i)}, f(x^{(i)}))h(x^{(i)}).$$

Define the gradient vector

$$g = \begin{bmatrix} \frac{\partial \ell}{\partial f}(Y^{(1)}, f(x^{(1)})) \\ \vdots \\ \frac{\partial \ell}{\partial f}(Y^{(N)}, f(x^{(N)})) \end{bmatrix}.$$

Also define the weak learner prediction vector

$$h_X = \begin{bmatrix} h(x^{(1)}) \\ \vdots \\ h(x^{(N)}) \end{bmatrix}.$$

Then the linearized risk change is proportional to

$$g^\top h_X.$$

Thus, to reduce empirical risk, gradient boosting chooses a weak learner whose predictions point approximately in the negative-gradient direction.

Definition 10.4.1. The negative gradient values

$$r_i = -\frac{\partial \ell}{\partial f}(Y^{(i)}, f(x^{(i)}))$$

are called pseudo-residuals or generalized residuals.

10.5 Gradient Boosting Algorithm

The boosting model has the additive form

$$f_M(x) = f_0(x) + \sum_{m=1}^{M} c_m h_m(x),$$

where each

$$h_m \in \mathcal{H}.$$

Algorithm 9 Gradient Boosting

1: Initialize

$$f_0 = \arg\min_{h \in \mathcal{H}} \frac{1}{N} \sum_{i=1}^{N} \ell(Y^{(i)}, h(x^{(i)})).$$

2: **for** $m = 1, \ldots, M$ **do**

3: Compute pseudo-residuals

$$r_i^{(m)} = -\frac{\partial \ell}{\partial f}(Y^{(i)}, f_{m-1}(x^{(i)})).$$

4: Fit a weak learner $h_m \in \mathcal{H}$ to the pseudo-residuals

$$r_1^{(m)}, \ldots, r_N^{(m)}.$$

5: Choose a step size c_m, for example by line search:

$$c_m = \arg\min_{c \in \mathbb{R}} \frac{1}{N} \sum_{i=1}^{N} \ell(Y^{(i)}, f_{m-1}(x^{(i)}) + c h_m(x^{(i)})).$$

6: Update

$$f_m(x) = f_{m-1}(x) + \nu c_m h_m(x),$$

where $0 < \nu \leq 1$ is a learning rate.

7: **end for**

Remark 10.5.1. The learning rate ν regularizes the boosting procedure. Smaller ν values usually require more boosting iterations but can improve generalization.

10.6 Least Squares Gradient Boosting

For squared error loss,

$$\ell(y, f) = (y - f)^2.$$

Then

$$\frac{\partial \ell}{\partial f} = -2(y - f).$$

Therefore, the negative gradient is

$$-\frac{\partial \ell}{\partial f} = 2(y - f).$$

Ignoring the constant factor 2, the pseudo-residuals are the ordinary residuals:

$$r_i = Y^{(i)} - f(x^{(i)}).$$

Thus, under squared error loss, gradient boosting repeatedly fits weak learners to the current residuals.

Proposition 10.6.1. *For squared error loss, gradient boosting performs functional gradient descent by fitting weak learners to residuals.*

Proof. The empirical risk is

$$R_N(f) = \frac{1}{N}\sum_{i=1}^{N}(Y^{(i)} - f(x^{(i)}))^2.$$

Differentiating with respect to the fitted value $f(x^{(i)})$ gives

$$\frac{\partial R_N}{\partial f(x^{(i)})} = -\frac{2}{N}(Y^{(i)} - f(x^{(i)})).$$

Thus the negative gradient direction is proportional to

$$Y^{(i)} - f(x^{(i)}),$$

which is exactly the residual. □

10.7 Weak Learner Complexity

The class $\mathcal{H}$ may contain decision stumps, shallow trees, splines, or other simple predictors. In tree-based boosting, the trees are usually kept shallow.

Remark 10.7.1. Boosting works by combining many weak learners. If each learner is too complex, the model may overfit quickly. If each learner is too weak, many iterations may be required.

10.8 AdaBoost

AdaBoost is a boosting algorithm for binary classification with labels

$$Y^{(i)} \in \{-1, 1\}.$$

It maintains weights on the training observations. Misclassified observations receive larger weights in later iterations, forcing future classifiers to focus on difficult cases.

Let

$$w_i^{(0)} = \frac{1}{N}, \qquad i = 1, \ldots, N.$$

At iteration j, fit a classifier

$$\hat{Y}_j(x) \in \{-1, 1\}$$

using the current observation weights.

The weighted error rate is

$$e_j = \frac{\sum_{i=1}^N w_i^{(j)} \mathbb{1}_{\{\hat{Y}_j(x^{(i)}) \neq Y^{(i)}\}}}{\sum_{i=1}^N w_i^{(j)}}.$$

Define

$$\alpha_j = \log\left(\frac{1 - e_j}{e_j}\right).$$

Update the weights by

$$w_i^{(j+1)} = w_i^{(j)} \exp\left(\alpha_j \mathbb{1}_{\{\hat{Y}_j(x^{(i)}) \neq Y^{(i)}\}}\right).$$

After J iterations, the final classifier is

$$\hat{Y}(x) = \text{sign}\left(\sum_{j=0}^{J-1} \alpha_j \hat{Y}_j(x)\right).$$

Algorithm 10 AdaBoost

1: Initialize $w_i^{(0)} = 1/N$ for $i = 1, \ldots, N$
2: **for** $j = 0, \ldots, J-1$ **do**
3: Fit a classifier $\hat{Y}_j$ using weights $w_i^{(j)}$
4: Compute weighted error

$$e_j = \frac{\sum_{i=1}^N w_i^{(j)} \mathbb{1}_{\{\hat{Y}_j(x^{(i)}) \neq Y^{(i)}\}}}{\sum_{i=1}^N w_i^{(j)}}$$

5: Compute

$$\alpha_j = \log\left(\frac{1 - e_j}{e_j}\right)$$

6: Update weights

$$w_i^{(j+1)} = w_i^{(j)} \exp\left(\alpha_j \mathbb{1}_{\{\hat{Y}_j(x^{(i)}) \neq Y^{(i)}\}}\right)$$

7: **end for**
8: Return

$$\hat{Y}(x) = \text{sign}\left(\sum_{j=0}^{J-1} \alpha_j \hat{Y}_j(x)\right)$$

Remark 10.8.1. If $e_j < 1/2$, then

$$\frac{1 - e_j}{e_j} > 1,$$

so

$$\alpha_j > 0.$$

Misclassified observations are multiplied by

$$e^{\alpha_j} > 1,$$

while correctly classified observations are unchanged. Therefore, later classifiers place more emphasis on examples that earlier classifiers missed.

10.9 AdaBoost and Exponential Loss

AdaBoost can be interpreted as minimizing exponential loss. For a classifier score function

$$F(x) = \sum_{j=0}^{J-1} \alpha_j \hat{Y}_j(x),$$

the prediction is

$$\hat{Y}(x) = \text{sign}(F(x)).$$

The exponential loss is

$$\ell(y, F(x)) = e^{-yF(x)}.$$

If $yF(x)$ is large and positive, the observation is correctly classified with high confidence and the loss is small. If $yF(x) < 0$, the observation is misclassified and the loss is large.

Remark 10.9.1. Gradient boosting is a general framework based on functional gradient descent. AdaBoost is a specific boosting method for binary classification that can be viewed as a special case involving exponential loss and adaptive reweighting.

10.10 Summary

Concept	Description
Empirical risk	Average loss over the training data
Weak learner	Simple model such as a decision stump or shallow tree
Boosting	Combines many weak learners into a stronger model
Gradient boosting	Adds learners in the negative-gradient direction of the empirical risk
Pseudo-residual	Negative derivative of the loss with respect to the current prediction
Least squares boosting	Fits weak learners to ordinary residuals

Learning rate	Shrinks each update to regularize the model
AdaBoost	Binary classification boosting method that upweights misclassified observations
Exponential loss	Loss function interpretation of AdaBoost

Part II

Scientific Computing

Chapter 11

Python Fundamentals

11.1 Assignment

Python uses assignment to bind a name to an object:

`name = object.`

```
x = 10
y = 3.14
name = "Jonathan"
```

Assignment does not copy the object by default. It creates another reference to the same object.

```
a = [1, 2, 3]
b = a
b.append(4)

print(a)  # [1, 2, 3, 4]
```

Here, both `a` and `b` refer to the same list.

Remark 11.1.1. For mutable objects such as lists and dictionaries, assignment creates an alias. For immutable objects such as integers, floats, strings, and tuples, this distinction is less visible because the object cannot be modified in place.

11.1.1 Multiple Assignment

Python supports multiple assignment:

```
x, y = 1, 2
```

This is useful for swapping variables:

```
x, y = y, x
```

11.1.2 Unpacking

Sequences can be unpacked into variables:

```
point = (3, 4)
x, y = point
```

The starred expression collects remaining values:

```
first, *middle, last = [1, 2, 3, 4, 5]

print(first)   # 1
print(middle)  # [2, 3, 4]
print(last)    # 5
```

11.2 Basic Data Types

Python objects have types. Common built-in types include:

Type	Description
`int`	Integer values such as `1`, `-5`, and `1000`
`float`	Floating-point values such as `3.14` and `-0.5`
`bool`	Boolean values `True` and `False`
`str`	Text strings
`list`	Mutable ordered sequence
`tuple`	Immutable ordered sequence
`dict`	Key-value mapping
`set`	Unordered collection of unique elements
`NoneType`	Represents absence of a value; written as `None`

11.2.1 Numbers

```
a = 10        # int
b = 3.5       # float

print(a + b)   # 13.5
print(a / 3)   # 3.333...
print(a // 3)  # 3
print(a % 3)   # 1
print(a ** 2)  # 100
```

11.2.2 Booleans

```
is_large = 10 > 5
is_equal = 10 == 5

print(is_large)  # True
print(is_equal)  # False
```

Boolean operators are:

```
x = 10

print(x > 0 and x < 20)
print(x < 0 or x == 10)
print(not x == 5)
```

11.2.3 Lists

Lists are mutable ordered sequences:

```
values = [10, 20, 30]

values.append(40)
values[0] = 5

print(values)  # [5, 20, 30, 40]
```

Indexing starts at zero:

```
values = [10, 20, 30, 40]

print(values[0])    # 10
print(values[-1])   # 40
print(values[1:3]) # [20, 30]
```

11.2.4 Tuples

Tuples are immutable ordered sequences:

```
point = (3, 4)

x, y = point
```

Remark 11.2.1. Tuples are useful when the grouped values should not be modified, such as coordinates, fixed records, or multiple return values.

11.2.5 Dictionaries

Dictionaries store key-value pairs:

```
student = {
    "name": "Jonathan",
    "program": "Applied Math and Statistics",
    "year": 2026
}

print(student["name"])
student["year"] = 2027
```

Use `get` to safely retrieve values:

```
major = student.get("major", "Not listed")
```

11.2.6 Sets

Sets store unique elements:

```
a = {1, 2, 3}
b = {3, 4, 5}

print(a | b)  # union
print(a & b)  # intersection
print(a - b)  # difference
```

11.3 Evaluation and Control Flow

Python evaluates expressions and returns values.

```
x = 2 + 3 * 4
print(x)  # 14
```

Parentheses can control evaluation order:

```
x = (2 + 3) * 4
print(x)  # 20
```

11.3.1 Conditional Statements

```
x = 10

if x > 0:
    print("positive")
elif x == 0:
```

```
    print("zero")
else:
    print("negative")
```

11.3.2 Loops

A `for` loop iterates over a sequence:

```
for value in [1, 2, 3]:
    print(value)
```

A `while` loop continues while a condition is true:

```
count = 0

while count < 3:
    print(count)
    count += 1
```

11.3.3 List Comprehensions

List comprehensions provide compact syntax for building lists:

```
squares = [x ** 2 for x in range(5)]
print(squares)
```

With a condition:

```
even_squares = [x ** 2 for x in range(10) if x % 2 == 0]
```

11.4 Functions

Functions package reusable computations:

```
def add(a, b):
    return a + b

result = add(2, 3)
```

11.4.1 Default Arguments

```
def greet(name, greeting="Hello"):
    return f"{greeting}, {name}"

print(greet("Jonathan"))
print(greet("Jonathan", greeting="Hi"))
```

11.4.2 Keyword Arguments

```
def power(base, exponent):
    return base ** exponent

print(power(base=2, exponent=3))
```

11.4.3 Variable-Length Arguments

```
def total(*values):
    return sum(values)

print(total(1, 2, 3, 4))
```

Keyword arguments can be collected with `**kwargs`:

```
def describe(**kwargs):
    for key, value in kwargs.items():
        print(key, value)

describe(name="Jonathan", program="AMS")
```

11.5 Timing Code

Timing code is useful for comparing implementations and identifying bottlenecks.

11.5.1 Using `time`

```
import time

start = time.time()

total = 0
for i in range(1_000_000):
    total += i

end = time.time()

print("Elapsed time:", end - start)
```

11.5.2 Using `timeit`

The `timeit` module is better for small snippets:

```python
import timeit

elapsed = timeit.timeit(
    stmt="sum(range(1000))",
    number=10000
)

print(elapsed)
```

Remark 11.5.1. Use `time.time()` for quick timing of larger code blocks. Use `timeit` for more reliable microbenchmarks.

11.6 Debugging Code

Debugging is the process of finding and fixing errors.

11.6.1 Common Error Types

Error	Meaning
`SyntaxError`	Python cannot parse the code
`NameError`	A variable or function name is not defined
`TypeError`	An operation is applied to an incompatible type
`ValueError`	Correct type, but invalid value
`IndexError`	Sequence index is out of range
`KeyError`	Dictionary key does not exist
`AttributeError`	Object does not have the requested attribute

11.6.2 Print Debugging

```python
def mean(values):
    print("values:", values)
    result = sum(values) / len(values)
    print("result:", result)
    return result
```

11.6.3 Assertions

Assertions check that expected conditions hold:

```python
def divide(a, b):
    assert b != 0, "b must be nonzero"
```

```
    return a / b
```

11.6.4 Using pdb

The built-in debugger can pause execution:

```
import pdb

def compute(values):
    total = sum(values)
    pdb.set_trace()
    return total / len(values)
```

Useful debugger commands include:

`n` for next, `s` for step, `c` for continue, `q` for quit.

11.7 String Manipulation

Strings are immutable sequences of characters.

```
text = "Introduction to Data Science"

print(text[0])
print(text[-1])
print(text[0:12])
```

11.7.1 Common String Methods

```
text = "  Python Programming  "

print(text.lower())
print(text.upper())
print(text.strip())
print(text.replace("Python", "Scientific"))
```

11.7.2 Splitting and Joining

```
sentence = "data,models,statistics"

parts = sentence.split(",")
print(parts)

joined = " | ".join(parts)
print(joined)
```

11.7.3 Formatted Strings

Formatted strings make interpolation readable:

```
name = "Jonathan"
score = 95.12345

message = f"{name} scored {score:.2f}"
print(message)
```

11.7.4 String Membership

```
text = "Bayesian statistics"

print("Bayesian" in text)
print("frequentist" in text)
```

11.8 Classes

Classes define custom object types. They bundle data and behavior.

```
class Student:
    def __init__(self, name, program):
        self.name = name
        self.program = program

    def introduce(self):
        return f"My name is {self.name} and I study {self.program}."

student = Student("Jonathan", "Applied Math and Statistics")
print(student.introduce())
```

11.8.1 The `self` Argument

Inside a class, `self` refers to the current object.

```
class Counter:
    def __init__(self):
        self.count = 0

    def increment(self):
        self.count += 1

counter = Counter()
counter.increment()
counter.increment()
```

```
11
12 print(counter.count)
```

11.8.2 Instance Attributes and Class Attributes

```
1  class Model:
2      model_type = "statistical model"  # class attribute
3
4      def __init__(self, name):
5          self.name = name              # instance attribute
```

11.9 Overloaded Operators

Python classes can define special methods to customize built-in behavior. These methods usually begin and end with double underscores.

11.9.1 String Representation

```
1  class Vector:
2      def __init__(self, values):
3          self.values = values
4
5      def __repr__(self):
6          return f"Vector({self.values})"
7
8  v = Vector([1, 2, 3])
9  print(v)
```

11.9.2 Addition

```
1  class Vector:
2      def __init__(self, values):
3          self.values = values
4
5      def __add__(self, other):
6          return Vector([
7              a + b
8              for a, b in zip(self.values, other.values)
9          ])
10
11     def __repr__(self):
12         return f"Vector({self.values})"
13
14 v1 = Vector([1, 2, 3])
```

```
v2 = Vector([4, 5, 6])

print(v1 + v2)
```

11.9.3 Scalar Multiplication

```
class Vector:
    def __init__(self, values):
        self.values = values

    def __mul__(self, scalar):
        return Vector([scalar * x for x in self.values])

    def __rmul__(self, scalar):
        return self.__mul__(scalar)

    def __repr__(self):
        return f"Vector({self.values})"

v = Vector([1, 2, 3])

print(v * 2)
print(2 * v)
```

11.9.4 Length and Indexing

```
class Dataset:
    def __init__(self, rows):
        self.rows = rows

    def __len__(self):
        return len(self.rows)

    def __getitem__(self, index):
        return self.rows[index]

data = Dataset([
    {"x": 1, "y": 2},
    {"x": 3, "y": 4}
])

print(len(data))
print(data[0])
```

Method	Behavior
`__repr__`	Object representation
`__add__`	Addition with `+`
`__sub__`	Subtraction with `-`
`__mul__`	Multiplication with `*`
`__truediv__`	Division with `/`
`__eq__`	Equality with `==`
`__lt__`	Less-than comparison with `<`
`__len__`	Length with `len(obj)`
`__getitem__`	Indexing with `obj[i]`

11.10 Decorators

A decorator is a function that modifies another function.

11.10.1 Basic Decorator

```
def announce(func):
    def wrapper():
        print("Starting function")
        result = func()
        print("Finished function")
        return result
    return wrapper

@announce
def greet():
    print("Hello")

greet()
```

The syntax

`@announce`

is equivalent to

`greet = announce(greet)`.

11.10.2 Decorators with Arguments

To decorate functions with arbitrary arguments, use `*args` and `**kwargs`:

```
def announce(func):
    def wrapper(*args, **kwargs):
```

```
        print("Starting function")
        result = func(*args, **kwargs)
        print("Finished function")
        return result
    return wrapper

@announce
def add(a, b):
    return a + b

print(add(2, 3))
```

11.10.3 Timing Decorator

```
import time

def timer(func):
    def wrapper(*args, **kwargs):
        start = time.time()
        result = func(*args, **kwargs)
        end = time.time()
        print(f"{func.__name__} took {end - start:.4f} seconds")
        return result
    return wrapper

@timer
def slow_sum(n):
    total = 0
    for i in range(n):
        total += i
    return total

slow_sum(1_000_000)
```

11.10.4 Using `functools.wraps`

A good decorator should preserve the original function metadata:

```
from functools import wraps

def timer(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        import time
        start = time.time()
        result = func(*args, **kwargs)
        end = time.time()
```

```
        print(f"{func.__name__} took {end - start:.4f} seconds")
        return result
    return wrapper
```

11.11 Class Decorators and Properties

The `@property` decorator makes a method behave like an attribute.

```
class Circle:
    def __init__(self, radius):
        self.radius = radius

    @property
    def area(self):
        return 3.14159 * self.radius ** 2

circle = Circle(2)
print(circle.area)
```

This allows computed attributes without calling a method explicitly.

11.12 Static Methods and Class Methods

A static method does not require access to the instance:

```
class MathUtils:
    @staticmethod
    def square(x):
        return x ** 2

print(MathUtils.square(5))
```

A class method receives the class itself as the first argument:

```
class Dataset:
    def __init__(self, values):
        self.values = values

    @classmethod
    def from_range(cls, n):
        return cls(list(range(n)))

data = Dataset.from_range(5)
print(data.values)
```

11.13 Summary

Topic	Main idea
Assignment	Names are bound to objects; mutable objects can have aliases
Data types	Python includes numbers, booleans, strings, lists, tuples, dictionaries, sets, and `None`
Evaluation	Expressions are evaluated using operator precedence and control flow
Timing	Use `time` for simple timing and `timeit` for small benchmarks
Debugging	Use error messages, print statements, assertions, and `pdb`
Strings	Strings support indexing, slicing, formatting, splitting, joining, and replacement
Classes	Classes bundle data and behavior into reusable object types
Overloaded operators	Special methods customize behavior of operators such as `+`, `*`, and indexing
Decorators	Decorators wrap functions or methods to modify behavior without changing the original code

Chapter 12

Advanced Python Topics for Data Science

12.1 Web Scraping and Regular Expressions

12.1.1 Web Scraping Overview

Web scraping extracts structured data from web pages. The standard workflow:

1. Send a request to a webpage
2. Parse the HTML
3. Extract relevant elements

12.1.2 Requests + BeautifulSoup

```
import requests
from bs4 import BeautifulSoup

url = "https://example.com"

response = requests.get(url)
html = response.text

soup = BeautifulSoup(html, "html.parser")

print(soup.title.text)
```

12.1.3 Finding Elements

```
# find first occurrence
link = soup.find("a")
```

```

# find all occurrences
links = soup.find_all("a")

# extract attributes
for link in links:
    print(link.get("href"))
```

12.1.4 Using CSS Selectors

```
elements = soup.select("div.classname > p")

for el in elements:
    print(el.text)
```

Remark 12.1.1. Websites may block scraping. Always check robots.txt and use rate limiting.

12.1.5 Regular Expressions

Regular expressions (regex) are used for pattern matching in strings.

```
import re

text = "Email: test@example.com"

match = re.search(r"\S+@\S+", text)
print(match.group())
```

12.1.6 Common Patterns

Pattern	Meaning
`.`	Any character
`\d`	Digit
`\w`	Word character
`*`	Zero or more
`+`	One or more
`?`	Optional
`^`	Start of string
`$`	End of string

12.1.7 Extraction Example

```
text = "Prices: $10, $25, $100"

prices = re.findall(r"\$\d+", text)
print(prices)
```

12.2 Parallel Computing

Parallel computing allows execution of multiple tasks simultaneously.

12.2.1 Multiprocessing

```
from multiprocessing import Pool

def square(x):
    return x ** 2

with Pool(4) as p:
    results = p.map(square, [1,2,3,4])

print(results)
```

12.2.2 Parallel Loops

```
from multiprocessing import Pool

def compute(x):
    return x * x

data = list(range(10))

with Pool() as pool:
    output = pool.map(compute, data)
```

12.2.3 Threading

Useful for I/O-bound tasks:

```
import threading

def task():
    print("Running task")

thread = threading.Thread(target=task)
```

```
thread.start()
thread.join()
```

Remark 12.2.1. Multiprocessing is better for CPU-bound tasks. Threading is better for I/O-bound tasks.

12.3 Priority Queues and Trees

12.3.1 Priority Queue (Heap)

Python provides a min-heap implementation:

```
import heapq

heap = []

heapq.heappush(heap, 5)
heapq.heappush(heap, 1)
heapq.heappush(heap, 3)

print(heapq.heappop(heap))  # 1
```

12.3.2 Custom Priority Queue

```
import heapq

tasks = []

heapq.heappush(tasks, (2, "medium"))
heapq.heappush(tasks, (1, "high"))
heapq.heappush(tasks, (3, "low"))

while tasks:
    priority, task = heapq.heappop(tasks)
    print(priority, task)
```

12.3.3 Binary Tree Implementation

```
class Node:
    def __init__(self, value):
        self.value = value
        self.left = None
        self.right = None
```

12.3.4 Traversal (Inorder)

```
def inorder(node):
    if node:
        inorder(node.left)
        print(node.value)
        inorder(node.right)
```

12.3.5 Binary Search Tree Insert

```
def insert(root, value):
    if root is None:
        return Node(value)
    if value < root.value:
        root.left = insert(root.left, value)
    else:
        root.right = insert(root.right, value)
    return root
```

12.4 PyTorch

PyTorch is a library for tensor computation and deep learning.

12.4.1 Tensors

```
import torch

x = torch.tensor([1.0, 2.0, 3.0])
y = torch.ones(3)

print(x + y)
```

12.4.2 Automatic Differentiation

```
x = torch.tensor(2.0, requires_grad=True)

y = x ** 2
y.backward()

print(x.grad)
```

12.4.3 Neural Network Example

```
import torch.nn as nn

model = nn.Sequential(
    nn.Linear(3, 5),
    nn.ReLU(),
    nn.Linear(5, 1)
)
```

12.4.4 Training Loop

```
optimizer = torch.optim.SGD(model.parameters(), lr=0.01)
loss_fn = nn.MSELoss()

for epoch in range(10):
    optimizer.zero_grad()
    predictions = model(x)
    loss = loss_fn(predictions, y)
    loss.backward()
    optimizer.step()
```

Remark 12.4.1. PyTorch builds computational graphs dynamically, which makes debugging and experimentation easier.

12.5 Pandas

Pandas is used for structured data manipulation.

12.5.1 DataFrame Creation

```
import pandas as pd

data = {
    "name": ["A", "B", "C"],
    "score": [90, 85, 88]
}

df = pd.DataFrame(data)
print(df)
```

12.5.2 Basic Operations

```
print(df.head())
print(df.describe())
print(df["score"])
```

12.5.3 Filtering

```
filtered = df[df["score"] > 85]
```

12.5.4 Adding Columns

```
df["passed"] = df["score"] > 80
```

12.5.5 GroupBy

```
df.groupby("passed")["score"].mean()
```

12.5.6 Merging DataFrames

```
df1 = pd.DataFrame({"id": [1,2], "x": [10,20]})
df2 = pd.DataFrame({"id": [1,2], "y": [100,200]})

merged = pd.merge(df1, df2, on="id")
```

12.5.7 Reading and Writing Files

```
df = pd.read_csv("data.csv")
df.to_csv("output.csv", index=False)
```

Remark 12.5.1. Pandas is central for data preprocessing pipelines and integrates well with NumPy, scikit-learn, and visualization libraries.

12.6 Summary

Topic	Key Idea
Web scraping	Extract structured data from HTML using requests and BeautifulSoup
Regular expressions	Pattern matching and text extraction
Parallel computing	Speed up computations via multiprocessing or threading
Priority queues	Efficient min-heap structure for ordered tasks
Trees	Recursive data structures for hierarchical data
PyTorch	Tensor computation and deep learning with automatic differentiation
Pandas	Tabular data manipulation and analysis

Chapter 13

Neural Networks

13.1 Architecture Overview

We define a parametric function

$$F_\theta : \mathbb{R}^3 \to \mathbb{R}^3$$

using a feedforward neural network with:

- Input layer: 3 nodes
- Hidden layer 1: 5 nodes
- Hidden layer 2: 4 nodes
- Output layer: 3 nodes

Each layer applies an affine transformation followed by a nonlinear activation:

$$\text{Layer Output} = \Psi\left(\alpha + \sum_j \beta_j v_j\right),$$

where Ψ is typically the ReLU activation

$$\Psi(t) = \max(0, t).$$

13.2 Layer Equations

Let $x \in \mathbb{R}^3$.

13.2.1 First Hidden Layer

$$h^{(1)} = \Psi(W^{(1)}x + b^{(1)}), \quad W^{(1)} \in \mathbb{R}^{5\times 3},\ b^{(1)} \in \mathbb{R}^5.$$

13.2.2 Second Hidden Layer

$$h^{(2)} = \Psi(W^{(2)}h^{(1)} + b^{(2)}), \quad W^{(2)} \in \mathbb{R}^{4\times 5},\ b^{(2)} \in \mathbb{R}^4.$$

13.2.3 Output Layer (Pre-Softmax)

$$z = W^{(3)}h^{(2)} + b^{(3)}, \quad W^{(3)} \in \mathbb{R}^{3\times 4},\ b^{(3)} \in \mathbb{R}^3.$$

13.3 Parameter Counting

Layer	Inputs	Nodes	Parameters per node	Total
Hidden 1	3	5	$1+3=4$	20
Hidden 2	5	4	$1+5=6$	24
Output	4	3	$1+4=5$	15
Total				59

Remark 13.3.1. Each node has a bias term plus weights for each input. Total parameters include all weights and biases.

13.4 Softmax Output Layer

Given logits z_0, z_1, z_2, define:

$$p_j = \frac{e^{z_j}}{\sum_{s=0}^{2} e^{z_s}}, \quad j = 0, 1, 2.$$

Then $p = (p_0, p_1, p_2)$ is a probability vector satisfying:

$$\sum_{j=0}^{2} p_j = 1.$$

13.5 Cross-Entropy Loss

Given training data $(x^{(i)}, Y^{(i)})$, define:

$$\mathcal{L}(\theta) = -\frac{1}{N}\sum_{i=1}^{N} \log p^{(i)}_{Y^{(i)}}(\theta).$$

This corresponds to maximum likelihood under a multinomial model.

13.6 Gradient Descent

We update parameters via:

$$\theta \leftarrow \theta - \varepsilon \nabla_\theta \mathcal{L}(\theta),$$

where $\varepsilon > 0$ is the learning rate.

13.6.1 Backpropagation Idea

Gradients are computed efficiently using the chain rule:

$$\frac{\partial \mathcal{L}}{\partial W^{(l)}} = \frac{\partial \mathcal{L}}{\partial h^{(l)}} \frac{\partial h^{(l)}}{\partial W^{(l)}}.$$

Each layer passes gradients backward through the network.

13.7 Interpretation

13.7.1 Decision Regions

The composition of affine transformations and nonlinearities produces a highly flexible function:

$$F_\theta(x) = W^{(3)} \Psi\Big(W^{(2)} \Psi(W^{(1)} x + b^{(1)}) + b^{(2)}\Big) + b^{(3)}.$$

This allows nonlinear partitioning of the input space into decision regions.

13.7.2 Training Dynamics

Training minimizes:

$$\mathcal{L}(\theta) = -\frac{1}{N} \sum_i \log p^{(i)}_{Y^{(i)}}(\theta).$$

A decreasing loss indicates improved predictive performance.

13.7.3 Probabilistic Output

The softmax ensures outputs can be interpreted as probabilities:

$$p_j = P(Y = j | x, \theta).$$

13.8 PyTorch Implementation

13.8.1 Model Definition

```
import torch
import torch.nn as nn

model = nn.Sequential(
    nn.Linear(3, 5),
    nn.ReLU(),
    nn.Linear(5, 4),
    nn.ReLU(),
    nn.Linear(4, 3)
)
```

13.8.2 Loss and Optimizer

```
loss_fn = nn.CrossEntropyLoss()
optimizer = torch.optim.SGD(model.parameters(), lr=0.01)
```

13.8.3 Training Loop

```
for epoch in range(100):
    optimizer.zero_grad()

    outputs = model(X)
    loss = loss_fn(outputs, y)

    loss.backward()
    optimizer.step()
```

Remark 13.8.1. In PyTorch, `CrossEntropyLoss` automatically applies softmax internally.

13.9 Mini-Batch Training

Instead of using all data at once, split into batches:

```
batch_size = 32

for epoch in range(10):
    for i in range(0, len(X), batch_size):
        xb = X[i:i+batch_size]
        yb = y[i:i+batch_size]

        optimizer.zero_grad()
        loss = loss_fn(model(xb), yb)
        loss.backward()
        optimizer.step()
```

13.10 Regularization

To prevent overfitting:

- Smaller networks
- Weight decay (L2 regularization)
- Dropout layers

```
optimizer = torch.optim.SGD(
    model.parameters(),
    lr=0.01,
    weight_decay=1e-4
)
```

13.11 Summary

Concept	Description
Node computation	$\Psi(\alpha + \sum_j \beta_j v_j)$
Activation	ReLU: $\Psi(t) = \max(0, t)$
Architecture	Layered affine + nonlinear transformations
Output	Softmax probabilities
Loss	Cross-entropy
Optimization	Gradient descent with backpropagation
Training	Iterative updates over epochs
Implementation	Efficiently handled via PyTorch